

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



NGUYEN THE KHOI

**A COOPERATIVE MULTI-AGENT DEEP
REINFORCEMENT LEARNING ANTI-JAMMING
METHOD IN BACKSCATTER COMMUNICATIONS
NETWORKS**

BACHELOR GRADUATE THESIS

Major: Computer science

HA NOI - 2025

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

NGUYEN THE KHOI

**A COOPERATIVE MULTI-AGENT DEEP
REINFORCEMENT LEARNING ANTI-JAMMING
METHOD IN BACKSCATTER COMMUNICATIONS
NETWORKS**

BACHELOR GRADUATE THESIS

Major: Computer science

Supervisor: Dr. Nguyen Ngoc Tan

Co-supervisor: MSc. Nguyen Thai Duong

HA NOI - 2025

AUTHORSHIP

I hereby declare that the work contained in this thesis is of my own and has not been previously submitted for a degree or diploma at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no materials previously published or written by another person except where due reference or acknowledgement is made.

Student

Nguyen The Khoi

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my supervisor, Dr. Nguyen Ngoc Tan, for his invaluable guidance, encouragement, and expertise throughout the development of this thesis. His insightful feedback and unwavering support were instrumental in shaping this research. I am also sincerely thankful to MSc. Nguyen Thai Duong for his mentorship and for inspiring me to explore the fascinating field of wireless communication systems, which laid the foundation for this work.

Finally, I am profoundly grateful to my family and friends for their constant encouragement, patience, and belief in me throughout this academic journey. This thesis would not have been possible without the collective support of all those mentioned.

Student

Nguyen The Khoi

ABSTRACT

ABSTRACT: Wireless communication plays a vital role in modern life but is highly vulnerable to jamming attacks due to its open-air nature. In backscatter communication networks, where energy efficiency is critical and devices are highly constrained, jamming poses an even greater challenge. This thesis proposes a cooperative anti-jamming method based on multi-agent deep reinforcement learning (MADRL) in ambient backscatter communication networks. By modeling each transmitter as an intelligent agent operating under a time-division multiple access (TDMA) scheme, agents learn cooperative strategies to avoid jamming and optimize long-term communication performance. Simulation results demonstrate that the proposed method significantly improves system resilience and throughput compared to traditional anti-jamming approaches, especially under dynamic jamming conditions. This work contributes a novel perspective on applying reinforcement learning to enhance the robustness of next-generation low-power wireless communication systems.

Keywords: *Anti-Jamming, Deep reinforcement learning, Backscatter*

TABLE OF CONTENTS

Authorship	
Acknowledgments	
Abstract	
Table of Contents	i
List of Figures	iii
CHAPTER 1. INTRODUCTION	1
1.1. Motivation	1
1.2. Objectives	2
1.3. Contributions of my thesis	2
1.4. Structure of the thesis	4
CHAPTER 2. THEORETICAL BACKGROUND	5
2.1. Wireless Network	5
2.2. Wireless Attack	7
2.2.1. Introduction	7
2.2.2. Evaluating Jamming Attack	8
2.3. Jamming Techniques	9
2.3.1. Proactive Jammers	9
2.3.2. Reactive Jammers	10
2.3.3. Function-Specific Jammers	10
2.3.4. Smart-Hybrid Jammers	11
2.4. Anti-jamming Techniques	12
2.4.1. Elementary Jamming Detection and Countermeasures	12
2.4.2. Detection and Countermeasure of Advanced Jamming	13
2.5. Ambient Backscatter Communications	13
2.5.1. Introduction	14
2.5.2. Description of Operating Principle	14
2.6. Machine Learning	16
2.7. Neural Network	17
2.7.1. Deep Learning	17

2.7.2. Neural Network	18
2.8. Reinforcement Learning	20
2.8.1. Introduction	20
2.8.2. Markov Decision Process	21
2.8.3. Policy and Value Functions	22
2.8.4. Bellman Equations	24
2.9. Solutions of MDP Problems	24
2.9.1. Dynamic Programming	24
2.9.2. Monte-Carlo Sampling	25
2.10. Q-learning	26
2.11. Deep reinforcement learning - DQN	27
CHAPTER 3. PROPOSED ANTI-JAMMING METHOD	30
3.1. System Model	30
3.1.1. Jamming model	30
3.1.2. System operation	31
3.2. Problem Formulation	33
3.2.1. State space	33
3.2.2. Action space	34
3.2.3. Immediate reward	35
3.3. Cooperative Multi-Agent Q-Learning and DQN for Anti-Jamming	38
3.3.1. Applying Cooperative Multi-Agent Q-Learning	38
3.3.2. Applying Cooperative Multi-Agent DQN	39
CHAPTER 4. Simulation Setup and Performance Evaluation	41
4.1. Experimental Setup Parameters	41
4.1.1. System Parameters	41
4.1.2. Q-learning and DQN Algorithm Parameters	42
4.2. Simulation Results	46
4.2.1. Convergence Comparison of the Q-learning and DQN Reinforce- ment Learning Algorithms	46
4.2.2. Performance Evaluation Under Different Number of Agent	48
4.2.3. Comparison of Different Learning Methods	52
Conclusions	60
References	63

LIST OF FIGURES

2.1	Hierarchy of Wireless Network Types by Coverage Area.	5
2.2	Types of jammers in wireless networks.	9
2.3	Decoder circuit for ambient backscatter signal.	16
2.4	Simplified visualization of neural network.	19
2.5	Visualization of the MDP.	21
2.6	Classical DQN architecture predicts Q-values.	29
3.1	System model.	31
3.2	Flowchart to express the possible actions of the transmitter.	36
3.3	Flowchart of Transmitter Decision Process in Multi-Agent System.	37
4.1	Convergence rates of Q-learning and deep Q-learning algorithms by total rewards.	46
4.2	Convergence rates of Q-learning and deep Q-learning algorithms by average rewards.	47
4.3	Packets loss ratio of Q-learning and deep Q-learning algorithms.	47
4.4	Packets loss ratio by transmission of Q-learning and deep Q-learning algorithms.	48
4.5	Convergence Rates of DQN Agents over 1M Iterations.	49
4.6	Convergence Rates of DQN Agents by average throughput over 1M Iterations.	50
4.7	Packets loss ratio of DQN agents.	50
4.8	Packets loss ratio by transmission of DQN agents.	51
4.9	Convergence Rates of QL Agents over 1M Iterations.	51
4.10	(a) Convergence Rates of QL Agents by average, (b) Packets loss ratio of QL agents.	52
4.11	Packets loss ratio by transmission of QL agents.	53
4.12	Convergence rates of Multi-Agent vs. Single-Agent systems.	54
4.13	Packets loss ratio of Multi-Agent vs. Single-Agent systems.	54
4.14	Packets loss ratio by transmission in Multi-Agent vs. Single-Agent systems.	55

4.15 Performance Comparison of Proposed Method vs. HTT Scheme Over 1M Iterations.	56
4.16 Convergence rates of the proposed method and HTT scheme under vary- ing jammer idle probabilities.	57
4.17 Packets loss ratio of the proposed method and HTT scheme under varying jammer idle probabilities.	57
4.18 Packets loss ratio by transmission under varying jammer idle probabili- ties for the proposed method and HTT scheme.	58
4.19 Comparison of Convergence Rates of the Proposed Solution vs. Fixed Agent TDMA.	59
4.20 Loss Ratio Comparison of the Proposed Solution vs. Fixed Agent TDMA.	59

List of Acronyms

ACK	Acknowledgment
BER	Bit Error Rate
BLE	Bluetooth Low Energy
CSMA	Carrier Sense Multiple Access
CTS	Clear-to-Send
FHSS	Frequency Hopping Spread Spectrum
IoT	Internet of Things
LTE	Long-Term Evolution
MADRL	Multi-Agent Deep Reinforcement Learning
MDP	Markov Decision Process
PDR	Packet Delivery Ratio
PSR	Packet Success Rate
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
QL	Q-Learning
RL	Reinforcement Learning
HTT	Harvest Then Transmit
RTS	Request-to-Send
SINR	Signal-to-Interference-plus-Noise Ratio
TDMA	Time Division Multiple Access
WLAN	Wireless Local Area Network
WMAN	Wireless Metropolitan Area Network

WSN Wireless Sensor Network

WWAN Wireless Wide Area Network

CHAPTER 1

INTRODUCTION

1.1. Motivation

Wireless communication systems have become a cornerstone of modern technology, enabling applications ranging from mobile networks to Internet of Things (IoT) devices. As wireless technologies continue to evolve and proliferate, ensuring the reliability and security of these communication systems is important. However, wireless networks are inherently susceptible to interference, both unintentional and malicious. Among these threats, jamming attacks pose a significant challenge, as they intentionally disrupt communication by overwhelming the channel with noise or interfering signals.

Traditional anti-jamming techniques, such as frequency hopping and spread spectrum methods, have been effective to some extent. However, these methods often lack adaptability and struggle to respond to dynamic or sophisticated jamming strategies. This limitation highlights the need for more advanced, intelligent, and adaptive solutions to combat jamming in wireless communication systems.

In recent years, advancements in artificial intelligence, particularly reinforcement learning (RL), have opened new methods for addressing complex problems in dynamic and uncertain environments. Deep Reinforcement Learning (DRL), which combines the representational power of deep learning with the decision-making capabilities of reinforcement learning, has shown immense potential in various fields, including robotics, gaming, and resource management. Leveraging DRL in wireless communication offers a promising approach to develop intelligent anti-jamming strategies that can learn and adapt in real-time.

This thesis focuses on designing and implementing a wireless anti-jamming method using Deep Reinforcement Learning. The proposed approach aims to enable wireless communication systems to dynamically detect, adapt to, and mitigate jamming attacks in a variety of scenarios. By exploring the interplay between jamming strategies and DRL-based defenses, this study seeks to contribute to the development of robust and resilient wireless communication systems.

The following sections of this thesis will provide a detailed background on jamming and anti-jamming techniques, discuss the theoretical underpinnings of Deep Reinforcement Learning, outline the methodology and experimental setup, present the results and analysis, and conclude with key findings and potential future research directions.

1.2. Objectives

The primary objectives of this thesis are to investigate the various types of jamming attacks that threaten wireless communication systems, focusing on their characteristics and the impact they have on communication reliability and security. The research aims to provide a comprehensive review of traditional anti-jamming techniques, such as frequency hopping and spread spectrum methods, by analyzing their strengths, limitations, and their suitability in addressing modern, dynamic, and sophisticated jamming threats.

A key objective is to explore the potential of Deep Reinforcement Learning (DRL) in enhancing the adaptability and effectiveness of anti-jamming strategies in wireless communication networks. This includes designing and developing a novel DRL-based anti-jamming method that can dynamically detect, adapt to, and mitigate jamming attacks in real-time, thus improving the resilience of wireless systems. The thesis also seeks to implement the proposed DRL-based solution in a simulation or real-world environment and evaluate its performance against traditional methods, measuring factors such as adaptability, efficiency, and effectiveness in counteracting various jamming strategies.

Through these efforts, the thesis aims to contribute to the advancement of more intelligent and secure wireless communication systems. Finally, the study will propose potential future research directions, offering insights into how DRL and other AI technologies can be further applied to strengthen wireless communication security and combat evolving jamming tactics.

1.3. Contributions of my thesis

This thesis introduces several novel contributions to the field of anti-jamming in backscatter communication networks through the development and evaluation of a cooperative multi-agent deep reinforcement learning (MADRL) framework. The key contributions are outlined as follows:

- 1. Development of a Cooperative Multi-Agent DQN Framework:** A novel anti-

jamming method is proposed, leveraging Deep Q-Network (DQN) techniques within a cooperative multi-agent setting. By modeling each transmitter as an intelligent agent operating under a Time Division Multiple Access (TDMA) scheme, the framework enables agents to collaboratively optimize energy harvesting, backscattering, and transmission strategies, enhancing system resilience against jamming attacks in energy-constrained environments. The proposed method achieves up to 1.8 packets per time slot in throughput, significantly outperforming single-agent systems (1.4 packets per time slot for DQN), demonstrating the advantage of multi-agent coordination in leveraging collective energy resources for effective anti-jamming.

2. **Comprehensive MDP Formulation for Multi-Agent Systems:** The anti-jamming problem is formulated as a Markov Decision Process (MDP) tailored for multi-agent cooperation. The formulation incorporates a continuous state space for DQN, capturing jammer states, TDMA schedules, and normalized data and energy queues, alongside a discretized state space for baseline Q-learning. This dual approach facilitates scalable and adaptive decision-making, addressing the dynamic nature of jamming in backscatter networks.
3. **Insights from Extensive Simulations:** Through detailed simulations, the proposed DQN-based method is shown to outperform traditional Q-learning, achieving higher average rewards and lower packet loss ratios. The comparison with the HTT scheme reveals a reduction in packet loss ratio, attributed to the DQN's ability to dynamically switch between backscatter and rate adaptation under varying jammer idle probabilities. Additionally, the multi-agent system demonstrates better transmission efficiency than single-agent systems, with lower packet loss by transmission despite higher overall packet loss due to queue overflow. The analysis of varying agent group sizes further reveals trade-offs between learning speed and queue overflow, providing practical insights for designing robust low-power wireless systems.
4. **Advancement of Backscatter Communication:** This work demonstrates the potential of MADRL to enhance throughput and mitigate jamming compared to both single-agent systems and traditional schemes like HTT. The proposed method's superior performance—achieving higher throughput and lower transmission-related

losses.

These contributions collectively advance the application of deep reinforcement learning in wireless communications, providing a robust and adaptive framework for combating jamming attacks. The proposed method not only addresses current challenges in backscatter networks but also lays the groundwork for future research into intelligent and secure wireless systems. To summarize, the primary contributions of this thesis are threefold. First, it introduces a cooperative multi-agent DQN framework tailored for anti-jamming in backscatter communication networks, integrating energy harvesting, backscatter, and TDMA scheduling to achieve up to 1.8 packets per time slot, surpassing single-agent performance. Second, it provides a comprehensive Markov Decision Process (MDP) formulation that captures the dynamics of multi-agent interactions under jamming. Third, the extensive simulation results offer valuable insights into the trade-offs of agent group size, multi-agent vs. single-agent performance, and the proposed method's superiority over HTT, highlighting the balance between learning speed, queue management, and transmission efficiency. These contributions advance the application of deep reinforcement learning in wireless communications.

1.4. Structure of the thesis

Chapter 2: Theoretical Background: This chapter presents the related work common to all methods and offers the necessary background knowledge for the following chapter.

Chapter 3: Proposed Anti-jamming Method: Chapter 3 presents the proposed anti-jamming method, leveraging Deep Q-Network (DQN) techniques with cooperation to address interference in wireless communication systems. It details the algorithm design, implementation, and adaptations for optimizing performance in jamming scenarios.

Chapter 4: Simulation Setup and Performance Evaluation: This chapter describes the simulation environment and methodology used to evaluate the proposed anti-jamming method. It includes details of the experimental setup, performance metrics, and comparative analysis to assess the effectiveness of the approach.

Conclusions: Summarize the key findings of the thesis, discuss the implications of the proposed method, and highlight its contributions. It also outlines limitations and suggests directions for future research.

CHAPTER 2

THEORETICAL BACKGROUND

This chapter provides an overview of the key theoretical concepts and technologies that form the foundation of the proposed wireless anti-jamming method using deep reinforcement learning.

2.1. Wireless Network

A wireless network is a type of communication network that allows devices to connect and communicate without the use of physical cables or wires. Instead, it uses radio waves, infrared signals, or other forms of electromagnetic radiation to transmit data between devices. Wireless networks are widely used for internet access, mobile communications, and device interconnectivity.

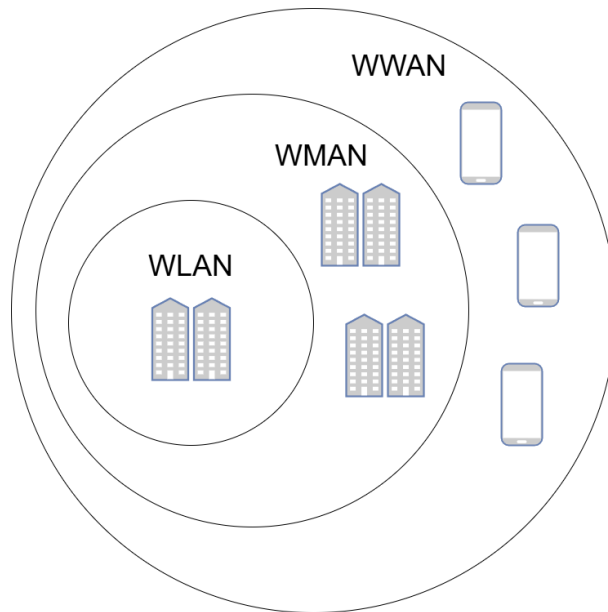


Figure 2.1. Hierarchy of Wireless Network Types by Coverage Area.

Type of wireless network:

- **Wireless Local Area Network (WLAN):** A WLAN connects devices within a confined area like a home, office, or campus using Wi-Fi (IEEE 802.11 standards). It allows mobile devices to communicate wirelessly without physical cables, offering flexibility and scalability in homes, schools, and businesses.

- **Wireless Metropolitan Area Network (WMAN):** A WMAN spans a city or metropolitan area, linking multiple WLANs. Wireless technologies like WiMAX and 4G LTE are commonly used, providing high-speed connectivity for businesses, public services, and municipal networks.
- **Wireless Wide Area Network (WWAN):** A WWAN covers vast geographic areas such as cities, countries, or continents. These networks rely on technologies like 4G, 5G, satellite, and microwave links to enable long-distance wireless communication, essential for mobile data, cloud services, and global connectivity.
- **Wireless Sensor Network (WSN):** A Wireless Sensor Network (WSN) consists of spatially distributed sensor nodes that collect and transmit data to monitor environmental conditions or physical processes. WSNs are typically used in applications like industrial monitoring, environmental sensing (e.g., weather, pollution), and agricultural systems (e.g., soil moisture, temperature). These networks often operate in low-power, low-bandwidth scenarios, making wireless technologies such as Zigbee, LoRaWAN, and Bluetooth Low Energy (BLE) ideal choices. WSNs can operate autonomously to collect and transmit data over time and are commonly used in Internet of Things (IoT) applications for real-time monitoring and decision-making.
- **Ad-hoc Networks:** Ad-hoc Networks are decentralized, self-configuring wireless networks that do not rely on a fixed infrastructure or central server. These networks are dynamically formed by devices that communicate with one another in a peer-to-peer fashion. They are often used in emergency situations, military operations, or other scenarios where traditional network infrastructure is unavailable. Mobile Ad-hoc Networks (MANETs) and Vehicular Ad-hoc Networks (VANETs) are common examples, where mobile devices or vehicles form temporary networks to exchange information. Ad-hoc networks are highly flexible, enabling rapid deployment and scalability, but they also present challenges related to security, routing, and network management.
- **Cellular Network:** A Cellular Network is a large-scale, wide-area wireless network used for mobile communication and data transfer. Cellular networks are divided into cells, each served by a base station or cell tower, which communicates

with mobile devices within its coverage area. These networks support voice communication, text messaging, and mobile data services such as internet browsing and app usage. The main wireless technologies used in cellular networks include 2G (GSM), 3G (UMTS), 4G (LTE), and the emerging 5G technology. 5G, in particular, promises faster speeds, lower latency, and increased device connectivity, enabling advanced applications such as autonomous vehicles, smart cities, and industrial automation.

2.2. Wireless Attack

2.2.1. Introduction

Radio jamming is defined as a deliberate act of interfering with the physical transmission and reception process in wireless communication. The attacker (jammer) emits radio signals on the same frequency band as the target network. The objective is to degrade network performance or completely block wireless communication between devices.

In a jamming attack, the jammer introduces interference energy into the wireless environment, disrupting legitimate communication in two ways:

- The jammer transmits strong interference signals, reducing the Signal-to-Interference-plus-Noise Ratio (SINR) at the receiver.
- Continuous jamming prevents the transmitter from accessing the channel, leading to a Denial-of-Service (DoS) attack. This is done by transmitting interference signals or fake packets to keep the channel busy, causing the transmitter to stop sending data until the channel is available again.

Characteristics of radio jamming attacks:

- **Intentional:** The attack is deliberately aimed at a specific target, unlike natural environmental interference.
- **Non-compliance with MAC protocols:** Jamming communication does not follow MAC protocols.
- **Scope:** The jammer can target a fixed or multiple frequencies.

2.2.2. Evaluating Jamming Attack

The following parameters reflect the impact of a jamming attack on wireless networks:

SINR (The Signal-to-Interference-plus-Noise Ratio) is given by:

$$\theta = \frac{P_R}{\phi P_J + \rho^2}, \quad (2.1)$$

where P_R is the received power from the transmitter at the receiver, P_J is the jammer's emitted interference power, ρ^2 is the variance of the additive white Gaussian noise, and $\phi \in [0, 1]$ is the channel attenuation factor.

A stronger jamming signal results in lower SINR, increasing the Bit Error Rate (BER), causing packet decoding errors, and reducing throughput and connection reliability.

Throughput: Defined as the average rate of successfully transmitted packets, calculated using Shannon's capacity formula:

$$C = B \log_2(1 + \text{SINR}), \quad (2.2)$$

where C is the channel capacity (bit/s), and B is the channel bandwidth (Hz). Throughput decreases when SINR is reduced by interference.

PSR (Packet Success Rate):

$$\text{PSR} = \frac{m}{n}, \quad (2.3)$$

where n is the number of packets intended for transmission, and $m \leq n$ is the number of packets successfully received.

Packet loss occurs due to channel congestion caused by interference, leading to packet drops either from a full buffer or long queueing times.

PDR (Packet Delivery Ratio):

$$\text{PDR} = \frac{q}{n}, \quad (2.4)$$

where n is the number of packets received and q is the number that pass CRC checking. PDR can also be calculated based on the number of ACKs received at the transmitter. If no packets are successfully received, then PDR is zero.

2.3. Jamming Techniques

Jamming disrupts wireless communication by intentionally creating radio interference, which can either keep the communication medium perpetually busy or corrupt received signals. While primarily targeting the physical layer, some jamming strategies may involve cross-layer attacks. This section categorizes jammers and explores their placement strategies for maximizing the affected area.

Jammers are malicious nodes that generate interference within a wireless network. Their effectiveness depends on factors like transmission power, location, and network influence. Jammers can be classified into elementary and advanced types based on their functionality. Elementary jammers are further divided into proactive and reactive, while advanced jammers include function-specific and smart-hybrid types.

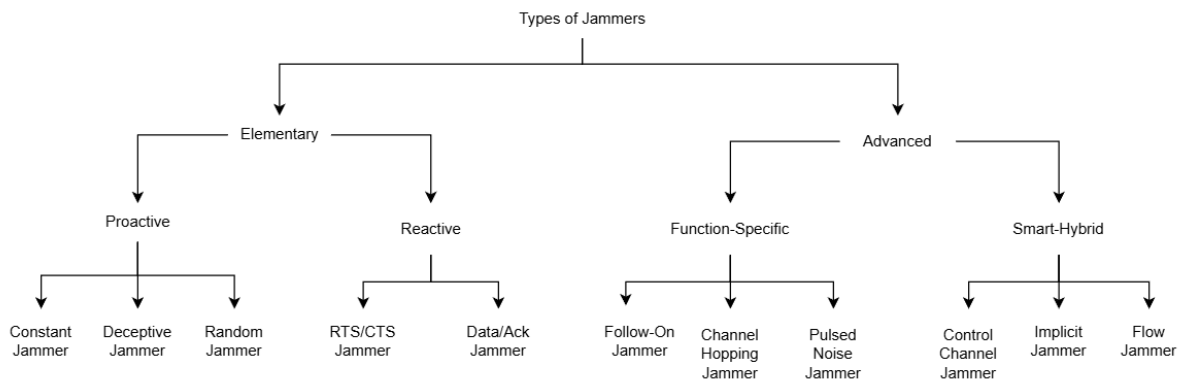


Figure 2.2. Types of jammers in wireless networks.

2.3.1. Proactive Jammers

Proactive jammers operate independently of network activity, continuously transmitting interfering signals.

- **Constant Jammers:** Emit continuous random bits without following the CSMA protocol, preventing network communication by occupying the medium. These are easy to launch and can damage network communications to the point that no one can communicate at any time but are energy inefficient and detectable.
- **Deceptive Jammers:** Transmit legitimate-looking packets to deceive nodes into staying in a receiving state. The detection of deceptive jammers presents significant obstacles, primarily because it transmits legitimate packets instead of random bits, but is also energy inefficient.

- **Random Jammers:** It intermittently transmits either random bits or regular packets into networks. It alternates between two modes: a sleep state and an active jamming state. The jammer remains inactive (sleeping) for a specific duration, then transitions to the jamming phase for a defined period, and subsequently returns to sleep. It conserves energy while also trading off jamming effectiveness for efficiency.

2.3.2. Reactive Jammers

Reactive jammers activate only during network activity, targeting message reception. They are more energy efficient than proactive types and harder to detect due to practical limitations in determining the packet delivery ratio (PDR).

- **RTS/CTS jammer** disrupts network communication by targeting the request-to-send (RTS) and clear-to-send (CTS) handshake. When it detects an RTS message being transmitted by a sender, it immediately begins jamming the channel. This interference prevents the receiver from sending a CTS reply, as the RTS packet becomes distorted. Consequently, the sender assumes the receiver is occupied with another transmission and refrains from sending data. Alternatively, the jammer may wait until the RTS is received and then jam the CTS reply from the receiver. In either case, the sender does not transmit data, and the receiver remains in a state of waiting for the data packet.
- **Data/ACK jammer** disrupts communication by corrupting either data packets or acknowledgment (ACK) messages. This jammer activates only when it detects the start of a data transmission. It may corrupt the data packets directly or wait until the data reaches the receiver before corrupting the ACK messages. Both scenarios result in retransmissions from the sender. In the first scenario, retransmissions occur because the receiver fails to correctly receive the data packets. In the second scenario, the sender, having not received the ACKs, assumes an issue at the receiver's end and retransmits the data packets.

2.3.3. Function-Specific Jammers

These jammers are tailored to specific tasks and can operate across single or multiple channels.

- **Follow-on jammer** rapidly hops across all available channels, switching thousands of times per second, and disrupts each channel briefly. When a transmitter detects the jamming and changes its channel, the jammer scans the entire frequency band to locate and jam the new channel. Alternatively, it may follow a pseudo-random frequency hopping sequence. This type of jammer is energy-efficient, targeting one channel at a time before moving to another. Its high hopping rate makes it particularly effective against anti-jamming strategies like frequency hopping spread spectrum (FHSS), which employs slower hopping rates.
- **Channel-hopping jammer** proactively switches between channels. It bypasses the MAC layer's CSMA algorithm, granting direct access to channels and enabling simultaneous jamming of multiple channels. During its discovery and vertex-coloring phases, it remains silent and undetectable by nearby devices. Once active, it attacks various channels at different times based on a predetermined pseudo-random sequence.
- **Pulsed-noise jammer** can switch between channels and target different bandwidths over time. Like a random jammer, it conserves energy by turning on and off according to its programmed schedule. However, unlike a basic proactive random jammer that disrupts only one channel, a pulsed-noise jammer can jam multiple channels simultaneously. It is also capable of targeting several channels at the same time.

2.3.4. Smart-Hybrid Jammers

Their primary objective is to amplify their disruptive impact on the targeted network while simultaneously conserving their own energy. By strategically deploying energy, they can significantly obstruct the communication bandwidth of the entire network or a substantial portion of it, especially in large-scale networks. Additionally, these jammers can operate in both proactive and reactive modes, making them hybrid in nature

- **Control channel jammers** disrupt multi-channel networks by targeting the control channel, which is essential for coordinating network activities. A random jammer attacking the control channel can significantly degrade network performance, while a continuous jammer can completely block network access. These attacks are often carried out by compromising a node within the network, allowing the attacker to access future control channel locations through the compromised nodes.

- **Implicit jamming attacks** not only disrupt the intended target but also create a denial-of-service state for other nodes in the network. These attacks exploit the rate adaptation algorithm in wireless networks, where the Access Point (AP) adjusts to weaker nodes by lowering its communication rate. When the attacker jams a node communicating with the AP, the rate adaptation mechanism increases the AP's focus on the jammed node, causing other nodes to experience reduced performance.
- **Flow-jamming attacks** involve multiple jammers distributed across the network, targeting packets to disrupt traffic flow. These attacks leverage information from the network layer and are resource-efficient for attackers. In centralized systems, the jammer calculates the minimum power needed to jam a packet and operates accordingly. In decentralized systems, jammers collaborate by sharing information with neighboring jammers to optimize their efficiency.

2.4. Anti-jamming Techniques

Jamming is a highly damaging form of DoS attack, making it crucial to implement effective detection and countermeasures. This section explores some techniques for detecting and mitigating both basic and advanced jamming.

2.4.1. *Elementary Jamming Detection and Countermeasures*

- **JAM Protocol:** For detecting and mapping jammed areas in wireless networks. It offers fast detection and rerouting to avoid affected regions, which can be effective in mobile or dynamic environments.
- **Ant System:** An evolutionary approach for detecting jamming at the PHY layer using network metrics like energy, hops, and packet loss. This system provides a decision-making process to confirm jamming and redirect traffic.
- **Channel Hopping:** A widely used countermeasure where nodes switch channels to avoid jamming. The method has variants such as proactive, reactive, and adaptive schemes, which can be selected depending on the network setup and jamming behavior.
- **Reactive Jamming Detection using BER:** Using the bit error rate (BER) to detect reactive jammers, particularly useful in networks where jamming causes disruptions in packet reception.

2.4.2. Detection and Countermeasure of Advanced Jamming

- **Hermes Node (Hybrid DSSS and FHSS):** The Hermes node combines Direct-Sequence Spread Spectrum (DSSS) and Frequency-Hopping Spread Spectrum (FHSS) to defend against fast-following jammers. DSSS spreads the signal over a wider bandwidth, making it appear as white noise to attackers, while FHSS rapidly changes frequencies (1 million hops per second) to avoid interference. The Hermes node uses 55 frequency channels for FHSS and 275 MHz of bandwidth for DSSS. A secret word, hard-coded into the network, is used to generate the frequency and PN codes, allowing nodes to synchronize and communicate securely.
- **Control Channel Attack Prevention:** To protect control channels from jamming, Lazos et al. propose using clusters, each with its own unique hopping sequence. If a jammer compromises a node, it can predict future hopping sequences, but this can be detected by computing the Hamming distance between the jammer's and the real sequence. The compromised node is identified, and the control channel is re-established using updated frequency hopping, minimizing the latency (evasion delay) and maximizing communication availability (evasion ratio).
- **MULEPRO (MULTi-channel Exfiltration Protocol):** It is a defense mechanism that activates when a jamming attack is detected. Nodes in normal mode communicate on a common channel, but when jamming occurs, they switch to exfiltration mode, utilizing multiple channels for communication to bypass the attack.

2.5. Ambient Backscatter Communications

Ambient Backscatter (AB) is an emerging wireless communication technology that differs from traditional methods by enabling devices to transmit data without using active RF components. Instead, it leverages existing radio signals in the environment as a communication medium. In the scenario considered, a wireless communication source—such as a mobile or television base station (denoted as BS)—broadcasts RF signals throughout its coverage area. Within this area, there is a transmitter and a receiver, both of which can receive the RF signals emitted by the BS. Since both devices are within the BS's range, they can simultaneously receive these signals. To send data to the receiver, the transmitter simply reflects and modulates the BS's signals (a process called backscattering) rather than generating its own.

2.5.1. Introduction

Ambient backscatter communication is a communication technique that allows two devices to transmit data to each other by leveraging radio frequency signals present in the surrounding environment. Ambient environmental signals are those readily available in the environment, such as noise signals, television signals, or mobile signals.

Compared to traditional wireless communication, this backscatter technique enables data transmission without energy-intensive processes like generating radio waves, resulting in significantly higher energy efficiency.

Compared to traditional backscatter techniques, such as RFID, which operates on the following principle: an RFID reader transmits high-power signals (1W) to nearby devices, such as passive RFID tags, which lack batteries and operate by harvesting energy from the RFID reader's signal. These tags then backscatter the signal to send information back to the reader, which decodes the backscattered signal to extract data. Ambient backscatter communication offers several advantages over RFID. First, since it utilizes signals already present in the environment, it does not require additional infrastructure to provide energy, such as RFID readers emitting high-power signals to power nearby devices. This reduces installation and maintenance costs, making systems like RFID challenging to deploy in outdoor environments or across large spatial areas. Second, its environmental impact is minimal since it consumes no additional energy beyond what is already available in the air. Finally, it enables direct communication between devices, unlike RFID systems where tags must communicate individually with the reader and cannot sense the transmission of nearby tags.

2.5.2. Description of Operating Principle

2.5.2.1. Packet Transmission Process at the Transmitter

The design of the transmitter in ambient backscatter communication is based on conventional backscatter communication techniques. Fundamentally, backscatter is achieved by altering the impedance of the antenna. Intuitively, when a wave encounters a boundary between two media with different impedances or density, the wave is reflected back. This principle applies whether it is a mechanical wave traveling through a string fixed to a wall or an electromagnetic wave encountering an antenna. By adjusting the antenna's impedance, the energy of the incoming RF wave can be controlled to be backscattered, thereby enabling information transmission.

The transmitter modulates and reflects ambient radio signals or interference signals using a load modulator. The input to the load modulator is a sequence of bits (0s and 1s) generated based on the packet. When the input bit is 0, the load modulator changes the impedance to Z_1 , placing the transmitter in a non-reflective state. When the input bit is 1, the load modulator changes the impedance to Z_2 , placing the transmitter in a reflective state. In this way, the transmitter can backscatter the packet to the receiver. Note that during backscatter, the transmitter can also harvest energy (in the non-reflective state), but the amount of energy harvested is relatively small and only sufficient for the operation of the backscatter circuit.

2.5.2.2. Packet Decoding Process at the Receiver

For the receiver to decode the backscattered signal, which carries information at a lower rate than the ambient environmental signals (specifically, interference signals and ambient signals), the decoding process can be described as follows. Suppose a digital receiver samples the signal at the Nyquist information rate (assuming an ADC is used). The received signal at the receiver is sampled as $y[n]$ according to:

$$y[n] = x[n] + \alpha B[n]x[n] + w[n], \quad (2.5)$$

where $x[n]$ is the sample of the ambient environmental signal (which may be an interference signal) received at the receiver, $w[n]$ is environmental noise, α is the complex attenuation of the backscattered signal, and $B[n]$ represents the data bits transmitted by the transmitter. If the transmitter sends information at a fractional rate, say $\frac{1}{N}$, then $B[Ni + j]$ are equal for $j \in \{1, \dots, N\}$. The receiver then calculates the average power of N received samples as follows:

$$\frac{1}{N} \sum_{i=1}^N |y[n]|^2 = \frac{1}{N} \sum_{i=1}^N |x[n] + \alpha Bx[n] + w[n]|^2, \quad (2.6)$$

where B takes the value 0 or 1 depending on the non-reflective or reflective state. Since $x[n]$ is uncorrelated with the noise $w[n]$, the above formula can be rewritten as:

$$\frac{1}{N} \sum_{i=1}^N |y[n]|^2 = |1 + \alpha B|^2 \frac{1}{N} \sum_{i=1}^N |x[n]|^2 + \frac{1}{N} \sum_{i=1}^N |w[n]|^2, \quad (2.7)$$

Denote $P = \frac{1}{N} \sum_{i=1}^N |x[n]|^2$ as the average power of the received interference signal (or ambient signal). Ignoring environmental noise, the average power received at the

receiver is $|1 + \alpha|^2 P$ and P when the transmitter is in the reflective ($B = 1$) and non-reflective ($B = 0$) states, respectively. Based on the difference between $|1 + \alpha|^2 P$ and P , the receiver can decode the data sent from the transmitter.

However, using an ADC to sample the signal at the receiver consumes significant energy. The following figure describes a network diagram that uses only analog components to decode the backscattered signal (see Figure 2.3).

Specifically, in the decoder shown in Figure 2.3, the backscattered signal is first smoothed by the “Average Envelope” circuit. Then, the “Compute Threshold” circuit outputs the low and high voltages of the smoothed signal. Subsequently, the “Comparator” circuit compares the signal with predefined thresholds to accurately extract the bits 0 and 1.

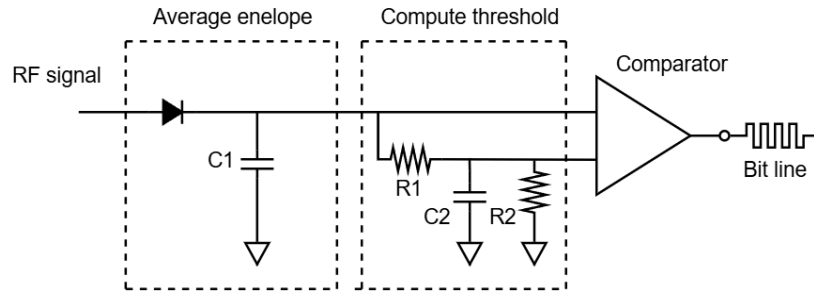


Figure 2.3. Decoder circuit for ambient backscatter signal.

2.6. Machine Learning

Machine learning (ML) is an approach to problem-solving where a machine learns a specific task (such as image classification, speech recognition, medical diagnosis, or resource allocation in networks) and measures performance (e.g., classification accuracy or performance loss) through data or experiences. The task typically involves a function that maps inputs to outputs. The core idea of data-driven ML is that there is a pattern between the inputs and the outcomes that cannot be defined mathematically, meaning the solution to the task—whether it’s decision-making or predicting an outcome—cannot be explicitly programmed. If the rules connecting inputs to outputs were known, a program could be written to solve the problem (e.g., using if-then-else conditions). Instead, an ML algorithm learns from the data, which specifies the correct output for each input. This results in a program that uses these data samples to solve the task. A typical data-driven ML architecture for classification problems includes a training module that optimizes the classifier using training data and provides a trained classifier to the classi-

fication module, which determines the output based on the input data. The training and classification modules work independently, with the training module being activated periodically, often running in the background while the classification module operates continuously. ML techniques can be categorized into three main types: supervised learning, unsupervised learning, and reinforcement learning.

- **Supervised Learning:** In this approach, given a dataset, a supervised learning algorithm predicts y based on the input-output relationship in D , and generalizes this mapping to new inputs x outside D . The dataset consists of input vectors $h(x)=y$ with high probability. A loss function, such as mean squared error, is used to evaluate the error between predicted and true labels. The dataset is typically divided into training, validation, and test sets, and the model is refined by revisiting the training process if the validation error is too high.
- **Unsupervised Learning:** These techniques are used to identify patterns or create internal representations of input data without requiring labeled data. Tasks like clustering, feature extraction, dimensionality reduction, and density estimation fall under unsupervised learning.
- **Reinforcement Learning (RL):** Unlike supervised or unsupervised learning, RL does not require a pre-existing dataset. An agent learns by interacting with its environment, gaining insights from the effects of its actions. This process mimics human learning, such as a child learning through exploration and repetition. The agent's goal is to maximize cumulative rewards by adjusting its behavior based on feedback from the environment.

2.7. Neural Network

2.7.1. Deep Learning

Deep learning (DL), a subset of machine learning (ML), has been introduced to address the limitations of traditional ML through the use of deep neural networks (DNNs). In deep learning, models are designed to automatically discover the essential features of data without requiring a predefined data structure. The core concept that makes deep learning a powerful tool in data science is its DNN architecture. The term "deep" refers to the number of layers in the DNN— the more layers, the deeper the network. Deep learning has been successfully applied in various everyday applications, including facial

and voice recognition, text translation, and intelligent driver assistance systems. Deep learning offers several advantages over traditional algorithms, including:

- No need for system modeling: Traditional optimization approaches require a well-defined system model to obtain optimal solutions, which may be challenging in complex systems like future wireless networks, where users' behaviors and network conditions are highly diverse and unpredictable. Even with a well-defined optimization problem, solving it can be difficult due to issues such as non-convexity and high dimensionality. Deep learning overcomes these challenges by being data-driven, allowing the optimal solution to be derived by training the DNN with sufficient data.
- Supports parallel and distributed algorithms: Deep learning often requires large volumes of labeled data to train DNNs effectively. This can be accelerated using parallel and distributed computing, leveraging multiple machines or systems for the training process. There are two main types of parallelism in deep learning: (i) model parallelism, where different layers of the model are trained on different devices, and (ii) data parallelism, where the same model is trained on different subsets of the data across multiple units.
- Reusability: A trained model in deep learning can be reused for different systems or problems, reducing the time and cost associated with training from scratch. For instance, models like AlexNet can be reused in new recognition tasks with minimal adjustments. Additionally, transfer learning allows a trained model to be applied to a related system, improving the training process with fewer data. This is beneficial because collecting training data is often costly and time-consuming.

2.7.2. Neural Network

Neural Networks are computational models inspired by the human brain's neural structure. They consist of layers of interconnected nodes (neurons) that process information. Each neuron receives an input, applies a transformation (activation function), and passes the output to the next layer of neurons.

How it works:

Neural networks consist of an interconnected set of nodes, organized across multiple layers. At the minimum, there are three types of layers:

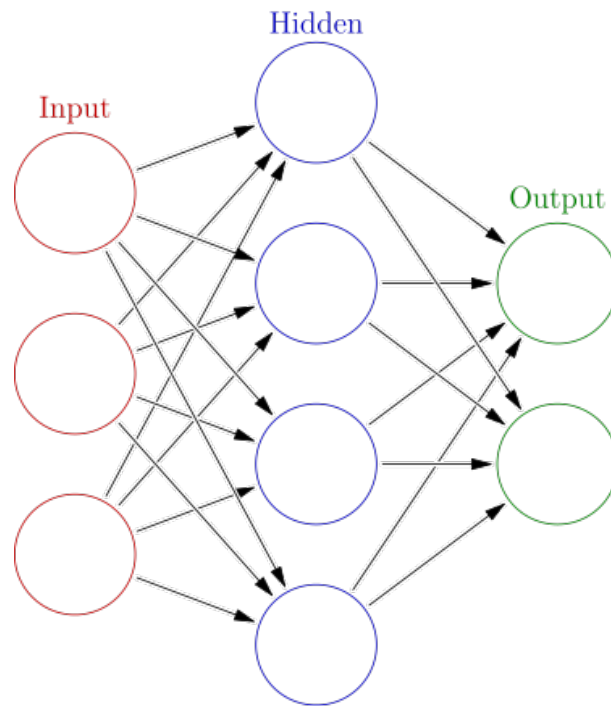


Figure 2.4. Simplified visualization of neural network.

- Input Layer: This layer receives the raw input data and feeds it into the network.
- Hidden Layers: One or more layers between the input and output layers where the actual processing takes place. These layers perform transformations on the data through complex computations.
- Output Layer: This layer produces the final output or prediction based on the processed data from the hidden layers.

While three layers are the minimum, neural networks can have multiple hidden layers, making them deep networks (as in deep learning). The more hidden layers, the greater the network's ability to model complex patterns and relationships in the data.

Each node, regardless of its position in the network, serves as a computational unit that processes the input it receives. When data is passed through the network, each node performs a weighted sum of its inputs and applies a mathematical function, such as an activation function, to this sum. The function typically applies a non-linearity to the data (e.g., ReLU or sigmoid), which enables the network to model more complex relationships.

If the processed output of a node exceeds a certain threshold, it passes the data to

the next layer. If the output does not surpass this threshold, the node does not transmit any information forward. This threshold is determined by the activation function and ensures that only relevant information is passed along through the network. The weights assigned to the inputs during processing are adjusted through training, allowing the network to optimize its performance over time.

In essence, the nodes in a neural network work together to process input data in a way that enables the network to learn and make predictions or decisions, with each layer gradually transforming the data to extract higher-level features or patterns.

Types of Neural Networks

- Feedforward Neural Networks (FNN): The simplest type, where information moves in one direction from input to output, without loops. Used for basic classification and regression tasks.
- Convolutional Neural Networks (CNN): Designed for processing grid-like data (such as images). CNNs are particularly effective in image recognition, object detection, and video analysis.
- Recurrent Neural Networks (RNN): Neural networks where connections form cycles, enabling them to process sequential data. They are suitable for tasks like language modeling, speech recognition, and time-series forecasting.
- Long Short-Term Memory Networks (LSTM): A type of RNN that can capture long-term dependencies in sequential data, addressing the vanishing gradient problem of traditional RNNs.
- Generative Adversarial Networks (GANs): Comprise two networks (generator and discriminator) that are trained simultaneously in a competitive manner. Used for generating new data (e.g., images, music).

2.8. Reinforcement Learning

2.8.1. Introduction

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with its environment. The goal of RL is for the agent to learn a policy that maximizes cumulative rewards over time. Unlike supervised learning,

where the model is trained on labeled data, RL involves learning from the outcomes of actions taken, with the agent receiving feedback in the form of rewards or penalties.

This approach is inspired by how humans and animals learn through trial and error. RL has been successfully applied in areas like robotics, game playing (e.g., AlphaGo), autonomous vehicles, and wireless communication systems. In the context of wireless networks, RL can optimize resource allocation, improve spectrum usage, and mitigate jamming or interference attacks by dynamically adapting to changing network conditions.

2.8.2. Markov Decision Process

MDP (Markov Decision Process) is a mathematical model used to describe the environment for reinforcement learning problems. MDP is a classical framework for sequential decision-making, where actions taken not only affect the immediate reward but also influence future situations, states, and rewards. The decision-making entity is called the agent, and the entity it interacts with is the environment. When the agent performs an action, the environment responds by changing its state and providing a numerical value representing the reward. The agent aims to maximize this reward over time through its choice of actions.

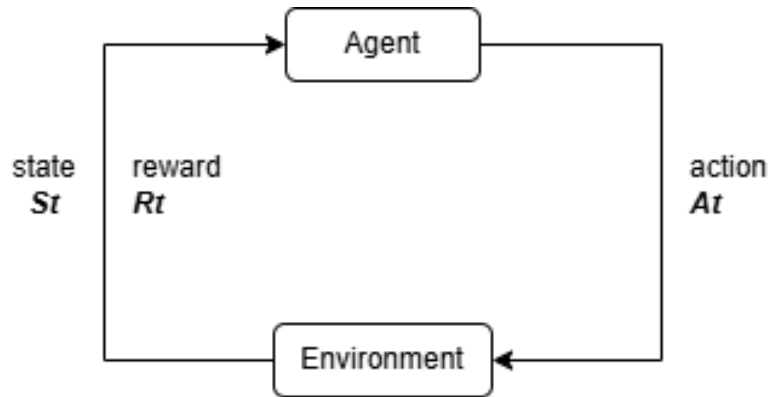


Figure 2.5. Visualization of the MDP.

To grasp the concept of an MDP, it is essential to understand the Markov property, which states that "the future is independent of the past, given the present." In other words, once the current state is identified, past experiences become irrelevant, as the current state provides all the necessary information to determine the next action.

$$P(s_{t+1} | s_t) = P(s_{t+1} | s_t, s_{t-1}, \dots, s_1), \quad (2.8)$$

Where:

- States (S): The set of all possible states the environment can be in. A state represents a particular situation or configuration of the system.
- Actions (A): The set of all possible actions the agent can take to influence the state of the environment.
- Transition Probability (P): A function that defines the probability of moving from one state to another, given a specific action.
- Reward (R): A numerical value given to the agent after taking an action in a particular state. The reward indicates the immediate benefit of the agent's action.
- Discount Factor (γ): A factor between 0 and 1 that represents the importance of future rewards compared to immediate rewards. A discount factor close to 1 prioritizes future rewards, while a factor close to 0 focuses on immediate rewards.

Based on the specific model described above, the Markov decision process can be formally expressed as follows: the agent and the environment interact through a sequence of discrete time steps $t = 0, 1, 2, \dots$. At each time step t , the agent receives the state of the environment $s \in S$ and uses this state to determine an action $a \in A$. In the next time step $t + 1$, the agent receives a numerical reward value $r_{t+1} \in R$, and the environment transitions to a new state s_{t+1} . The state transition probability function P captures the dynamics of the environment in the MDP.

Reinforcement learning provides a solution to the MDP problem when the parameters of the MDP model are not explicitly defined.

2.8.3. Policy and Value Functions

The agent's behavior is guided by a policy, which dictates how the agent responds to each state at every decision epoch. Policies can be categorized into two types: deterministic and stochastic, based on the chosen model. A deterministic policy is defined as a function $\pi(s_t) = a_t$, where $s_t \in S$ and $a_t \in A$, mapping each state directly to a specific action. In contrast, a stochastic policy represents a probability distribution over the action space A , such that $\pi(a_t | s_t) = p_i \in [0, 1]$, providing the probabilities of selecting different actions given the current state. The policies are evaluated through the expected discounted return expressed as follows:

$$R_t = r_{t+1} + \gamma r_{t+2} + \cdots = \sum_{k=0}^T \gamma^k r_{t+k+1}, \quad (2.9)$$

Where r_{t+k+1} is the immediate reward received at time step $t + k + 1$.

The discount factor γ , included in the MDP framework, can be adjusted in practice to improve the learning process. A low discount factor encourages the agent to focus on maximizing immediate rewards, resulting in more short-sighted, greedy behavior. Conversely, a discount factor close to 1 places more emphasis on future rewards, treating them almost as equally important as immediate ones.

For episodic tasks, where a terminal state is reached, γ can be set to 1. However, for continuous tasks with infinite learning trajectories, γ must be strictly less than 1.

While it is theoretically proven that an optimal policy (π^*) exists, which performs as well as or better than any other policy, deriving this optimal policy analytically is challenging. Therefore, reinforcement learning algorithms are designed to iteratively search for optimal or near-optimal policies that maximize the expected return.

A value-function estimates how good a state is for the agent based on the expected reward. The value function, $V_\pi(s)$, is defined in relation to a specific policy π , as the future rewards depend on the actions that the policy dictates for the agent to take.

$$V_\pi(s) = \mathbb{E}_\pi\{R_t \mid s_t = s\} = \mathbb{E}_\pi\left\{\sum_{k=0}^T \gamma^k r_{t+k+1} \mid s_t = s\right\}, \forall s \in \mathcal{S}, \quad (2.10)$$

Where $\mathbb{E}_\pi\{\}$ denotes the expected value given a policy π .

Similarly, we can define the value of taking an action a in state s with respect to policy π as follows:

$$Q^\pi(s, a) = \mathbb{E}_\pi\{R_t \mid s_t = s, a_t = a\} = \mathbb{E}_\pi\left\{\sum_{k=0}^T \gamma^k r_{t+k+1} \mid s_t = s, a_t = a\right\}, \quad (2.11)$$

where $Q_\pi(s, a)$ is the state-action value function for policy π .

The value function can be expressed as a sum of state-action values over all possible actions in the current state, weighted by the probability of the action being chosen (stochastic policy):

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(s, a) Q^\pi(s, a). \quad (2.12)$$

The value functions V_π and Q_π are estimated from previous experiences sampled from the environment.

2.8.4. Bellman Equations

The Bellman equations are recursive equations used to compute the value of a state or the expected return for a given policy. The Bellman equation provides a way to break down a complex decision-making problem into simpler subproblems by expressing the value of a state as the sum of immediate rewards and the expected value of future states. For a given policy, the Bellman equation for the value function is:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(s, a) \sum_{s' \in \mathcal{S}} P(s'|s, a) [r(s, a, s') + \gamma V^\pi(s')]. \quad (2.13)$$

The Bellman equation for the action-value function is:

$$Q^\pi(s, a) = \sum_{s' \in \mathcal{S}} P(s'|s, a) [r(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi(s', a') Q^\pi(s', a')]. \quad (2.14)$$

The Bellman equations are the foundation of many reinforcement learning algorithms, including dynamic programming, Q-learning, and value iteration.

2.9. Solutions of MDP Problems

2.9.1. Dynamic Programming

Dynamic Programming (DP) is a family of algorithms used to compute exact solutions for Markov Decision Processes (MDPs) when the environment's transition dynamics $P(\cdot|s, a)$ are known. DP methods iteratively compute value functions (policy evaluation) and use them to improve policies (policy improvement).

2.9.1.1. Policy Evaluation

Policy evaluation computes the state-value function $V^\pi(s)$ for a given policy π . This is achieved through iterative application of the Bellman equation:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [r(s, a, s') + \gamma V_k(s')], \quad \forall s \in \mathcal{S}, \quad (2.15)$$

where $V_0(s)$ is initialized arbitrarily (with $V_0(s) = 0$ for terminal states). The sequence $\{V_k\}$ converges to V^π as $k \rightarrow \infty$, provided $\gamma < 1$ or all policies lead to termination. Convergence is typically assessed by checking $\max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)| < \epsilon$, where ϵ is a small threshold. This process is known as iterative policy evaluation.

2.9.1.2. Policy Improvement

Policy improvement constructs a better policy π' from an existing policy π . The policy improvement theorem states that if $V^{\pi'}(s) \geq V^\pi(s)$ for all $s \in \mathcal{S}$, then π' is at least as good as π . A new policy is derived by selecting actions that maximize the action-value function:

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [r(s, a, s') + \gamma V^\pi(s')]. \quad (2.16)$$

Policy iteration alternates between policy evaluation and policy improvement. Starting with an initial policy π_0 , the algorithm evaluates V^{π_k} and improves it to obtain π_{k+1} , repeating until the policy stabilizes.

2.9.1.3. Value Iteration

Value iteration combines policy evaluation and improvement into a single update to compute the optimal state-value function $V^*(s)$:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [r(s, a, s') + \gamma V_k(s')], \quad \forall s \in \mathcal{S}. \quad (2.17)$$

The process repeats until convergence ($\max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)| < \epsilon$). The optimal policy is then derived as:

$$\pi(s) = \arg \max_a \sum_{s'} P(s'|s, a) [r(s, a, s') + \gamma V^*(s')]. \quad (2.18)$$

2.9.2. Monte-Carlo Sampling

A key limitation of DP is its reliance on known environment dynamics, which may be unavailable in practice. Monte Carlo (MC) sampling addresses this by estimating value functions using sampled trajectories, eliminating the need for $P(\cdot|s, a)$.

2.9.2.1. Procedure

MC sampling estimates the state-value function $V^\pi(s)$ or action-value function $Q^\pi(s, a)$ as follows:

1. Start from an initial state s_0 .
2. Generate a trajectory $\tau_t = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T, a_T, r_{T+1}, s_{T+1})$ using policy π .

3. Compute the discounted cumulative reward $R_t^{(e)} = \sum_{k=0}^T \gamma^k r_{t+k+1}$ for time step t .
4. Repeat for M trajectories and estimate:

$$V^\pi(s) \approx \frac{1}{M} \sum_{t=1}^M R_t^{(e)}. \quad (2.19)$$

In practice, incremental updates are used:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha (R_t - V^\pi(s)), \quad (2.20)$$

$$Q^\pi(s, a) \leftarrow Q^\pi(s, a) + \alpha (R_t - Q^\pi(s, a)), \quad (2.21)$$

where $\alpha \in (0, 1)$ is a step-size parameter.

2.9.2.2. Limitations and Exploration

MC methods are well-suited for episodic tasks with finite trajectories but struggle with continuous tasks due to challenges in reward averaging. Additionally, MC relies heavily on exploration to avoid convergence to suboptimal policies. Greedy action selection can trap the agent in local minima, while fully random selection evaluates a random policy rather than π . The exploration-exploitation trade-off is managed by favoring exploration early in learning and exploitation later, often through techniques like ϵ -greedy action selection.

2.10. Q-learning

Q-learning is an off-policy, model-free reinforcement learning (RL) method that enables an agent to learn optimal actions by interacting with an environment. Unlike Monte-Carlo methods, which rely on averaging returns over entire episodes, Q-learning uses Temporal Difference (TD) learning to update estimates based on immediate rewards and the expected future rewards, making it suitable for both episodic and infinite tasks. In Q-learning, the agent selects actions to maximize the Q-function, $Q^\pi(s, a)$, which represents the expected cumulative reward for taking action a in state s and following policy π . The update rule for Q-learning is:

$$Q^\pi(s, a) \leftarrow Q^\pi(s, a) + \alpha \left(r(s, a, s') + \gamma \max_{a'} Q^\pi(s', a') - Q^\pi(s, a) \right), \quad (2.22)$$

where α is the learning rate, γ is the discount factor, $r(s, a, s')$ is the immediate reward, and $\max_{a'} Q^\pi(s', a')$ reflects the greedy action selection in the next state s' . The term

$\delta = r(s, a, s') + \gamma \max_{a'} Q^\pi(s', a') - Q^\pi(s, a)$ is the TD error, quantifying the difference between predicted and actual rewards.

Q-learning employs an ϵ -greedy strategy for action selection to balance exploration and exploitation:

$$a = \begin{cases} \arg \max_a Q(s, a) & \text{with probability } 1 - \epsilon, \\ \text{random action} & \text{with probability } \epsilon. \end{cases} \quad (2.23)$$

The algorithm initializes $Q(s, a) = 0$ for all states and actions, iteratively updates the Q-values, and converges to the optimal policy under appropriate conditions, such as a decaying learning rate α satisfying:

$$\sum_{t=0}^{\infty} \alpha_t(s, a) = \infty, \quad \sum_{t=0}^{\infty} \alpha_t^2(s, a) < \infty. \quad (2.24)$$

Q-learning's key advantage is its ability to update Q-values without waiting for episode completion, enabling efficient learning. However, it relies on estimates that may initially be inaccurate, requiring multiple iterations to stabilize.

Algorithm 2.1 Tabular Q-learning

```

1: Initialize  $Q_0(s, a) = 0 \forall s, a$ 
2: Get initial state  $s_0$ 
3: while not converged,  $k = 1, 2, \dots$  do
4:   Choose action  $a$ , get next state  $s'$  and reward  $r$ 
5:   if  $s'$  is terminal then
6:      $y_{\text{target}} = r$ 
7:     Sample new initial state  $s'$ 
8:   else
9:      $y_{\text{target}} = r + \gamma \max_{a'} Q_k(s', a')$ 
10:  end if
11:   $Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha y_{\text{target}}$ 
12:   $s \leftarrow s'$ 
13: end while

```

2.11. Deep reinforcement learning - DQN

*Deep Q-Network

Deep Q-Network (DQN) is a reinforcement learning (RL) approach that employs neural networks as function approximators to handle tasks with large or continuous state and action spaces. Unlike supervised learning, RL faces challenges such as correlated transition data and non-stationary target values, which can destabilize neural network training. DQN, introduced by Mnih et al., addresses these issues through two key innovations: **experience replay** and a **target network**.

Experience Replay: To mitigate the impact of correlated transitions, DQN stores experiences (state, action, reward, next state) in a replay buffer. Minibatches are randomly sampled from this buffer for training via Stochastic Gradient Descent (SGD), reducing correlation and improving stability. Old transitions are replaced when the buffer is full.

Target Network: To address non-stationary targets in the Q-learning update $r(s, a, s') + \gamma \max_{a'} Q(s', a')$, DQN uses a separate target network $Q_{\theta'}$ with parameters θ' , identical in structure to the primary Q-network Q_{θ} . The target network is updated less frequently (e.g., every few thousand steps) with the latest parameters θ , ensuring more stable target values.

The DQN algorithm optimizes a loss function, typically the squared loss or Huber loss for robustness against outliers, using optimizers like SGD, Adam, or RMSProp. The Huber loss is defined as:

$$L_{\delta}(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| < \delta, \\ \delta(|x| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases} \quad (2.25)$$

DQN's architecture, often involving convolutional and fully connected layers, predicts Q-values for each action. Despite its effectiveness, DQN learns slowly, requiring many samples due to high sampling complexity. Uniform sampling from the replay buffer may delay the use of recent transitions or overuse outdated ones, and infrequent target network updates can initially lead to suboptimal policies. Advanced methods like DDPG use smooth Polyak updates for the target network:

$$\theta' = \tau\theta + (1 - \tau)\theta', \quad \tau \ll 1. \quad (2.26)$$

In applications like wireless communications, DQN excels in discrete action spaces, e.g., dynamic multi-channel access, outperforming traditional methods. For continuous

spaces, discretization is often applied to adapt DQN.

Figure 2.6. Classical DQN architecture predicts Q-values.

Algorithm 2.2 Deep Q-Network (DQN) with Experience Replay

- 1: Initialize replay buffer B to store transitions
 - 2: Initialize randomly the Q-function Q_θ and target network $Q_{\theta'}$ such that $\theta = \theta'$
 - 3: Observe initial state s_1
 - 4: **for** $t = 1, \dots, T$ **do**
 - 5: Select action a_t based on the behavior policy derived from $Q_\theta(s_t, a_t)$ (e.g., ϵ -greedy)
 - 6: Observe new state s_{t+1} and reward r_t
 - 7: Store transition (s_t, a_t, r_t, s_{t+1}) in replay memory B
 - 8: Sample minibatch of transitions (s_j, a_j, r_j, s_{j+1}) from B
 - 9: Set $y_j = \begin{cases} r_j & \text{if } s_{j+1} \text{ is terminal,} \\ r_j + \gamma \max_{a'} Q_{\theta'}(s_{j+1}, a') & \text{otherwise.} \end{cases}$
 - 10: Optimize $L(\theta) = L(Q_\theta(s_j, a_j) - y_j)$ using SGD (or its variants)
 - 11: Every C steps, reset $\theta' = \theta$
 - 12: **end for**
-

CHAPTER 3

PROPOSED ANTI-JAMMING METHOD

In this chapter, the anti-jamming problem in backscatter communication networks is modeled as an optimization problem using a Markov Decision Process (MDP), aiming to maximize long-term throughput. A cooperative multi-agent approach using Q-learning and Deep Q-Networks (DQNs) under a TDMA schedule is proposed to solve the MDP optimization problem. This enables the communication channel not only to resist jamming attacks but also to exploit them.

3.1. System Model

In this section, we describe the system model for our cooperative multi-agent deep reinforcement learning (DRL) anti-jamming method in backscatter communication networks. The model is designed to address the challenges posed by jamming attacks in a wireless network, leveraging ambient backscatter and energy harvesting capabilities to enhance communication resilience.

***System Model and Assumptions**

We consider a wireless network comprising a receiver, a set of N transmitter agents, and a jammer, as illustrated in Figure 3.1. The agents are positioned within a circular range around the jammer, which attempts to disrupt communications between the transmitters and the receiver. Each agent is equipped with energy harvesting and ambient backscatter circuits, enabling it to harvest energy from jamming signals and backscatter data under attack. The system operates in discrete time slots using a Time Division Multiple Access (TDMA) scheduling mechanism, where each time slot is assigned to a specific agent to act as the primary transmitter, while others perform supportive actions.

3.1.1. Jamming model

The jammer aims to degrade the effective signal-to-interference-plus-noise ratio (SINR) at the receiver by transmitting interference signals. The SINR at the receiver for an agent i is given by:

$$\theta_i = \frac{P_{R,i}}{\phi P_J + \rho^2}, \quad (3.1)$$

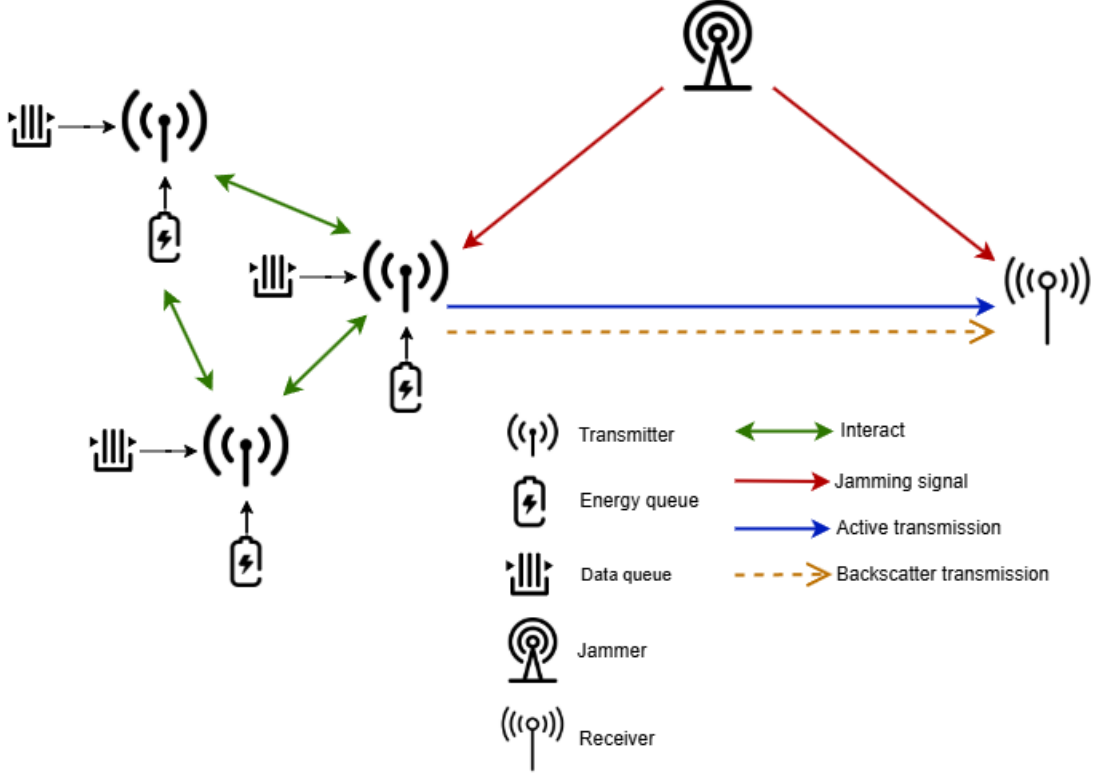


Figure 3.1. System model.

where $P_{R,i}$ is the received power from agent i at the receiver, P_J is the jamming power, $\phi \in [0, 1]$ is the channel attenuation factor, and ρ^2 is the variance of additive white Gaussian noise.

The jammer operates in one of two states: idle ($j = 0$) or active ($j = 1$). When active, the jammer attacks with a power level P_n^J selected from a discrete set $\mathbf{P}_J = \{P_0^J, P_1^J, \dots, P_N^J\}$, where P_0^J corresponds to the idle state ($P_0^J = 0$). The probability of selecting power level P_n^J is x_n , satisfying:

$$\mathbf{J}_s = \left\{ \mathbf{x} = \langle x_0, x_1, \dots, x_N \rangle \mid \sum_{n=0}^N x_n = 1 \right\}. \quad (3.2)$$

3.1.2. System operation

The system operates in discrete time slots, with each slot assigned to one agent via TDMA scheduling, i.e., at time slot t , agent $i = t \bmod N$ is the primary transmitter. Each agent maintains a data queue of size $D = D$ and an energy queue of size $E = E$. Data packets arrive at each agent's queue according to a Poisson process with rate λ packets per time slot. If the data queue is full, arriving packets are dropped.

When the jammer is idle, the primary agent can:

- Stay idle
- Actively transmit up to $\hat{d}_t$ packets, consuming e_t unit of energy per packet, provided there is sufficient energy and data.

When the jammer is active, the primary agent can:

- Harvest energy from the jamming signals, gaining e_n^J units of energy based on the jamming power P_n^J , with $e_n^J \in \{e_0^J, \dots, e_N^J\}$ corresponding to jamming power.
- Backscatter the jamming signals, transmitting up to $b^\dagger$ packets, with a maximum successful rate of $\hat{d}_n^J \in \{\hat{d}_0^J, \dots, \hat{d}_N^J\}$ corresponding to jamming power; if $b^\dagger > \hat{d}_n^J$, the excess packets are lost.
- Perform rate adaptation, $\mathbf{r} = \{r_1, \dots, r_m, \dots, r_M\}$ denote the set of available transmission rates that the transmitter can choose to transmit data when the jammer attacks the channel. At each rate r_m transmitting a maximum of $\hat{d}_m^r$ packets, consuming e_t unit of energy per packet, with losses if the selected rate exceeds the channel's capacity under jamming.
- * If the primary agent happens to not be transmitting, it can check other agents to see if they have the best action for transmission and select one agent with the highest reward to transmit. The others will act as non-primary agents.

Non-primary agents (i.e., agents $i \neq t \bmod N$) can only:

- Stay idle
- Harvest energy

The agents cooperate to maximize the total throughput by learning to balance energy harvesting, backscattering, and active transmissions, adapting to the jammer's strategy.

3.2. Problem Formulation

To address the dynamics and uncertainty of the jamming environment, we formulate the anti-jamming problem as a Markov Decision Process (MDP) for the multi-agent system. The MDP is defined by the tuple $\langle \mathcal{S}, \mathcal{A}, r \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and r is the immediate reward function. The goal is to find an optimal policy π^* that maximizes the average long-term throughput across all agents.

3.2.1. State space

The system state at time slot t is defined to capture the dynamics of the multi-agent system, enabling cooperative decision-making. For the deep Q-learning (DQN) algorithm, which is the primary focus of this work, the state is represented as a continuous vector that includes the states of all agents. Specifically, the state is defined as:

$$\mathbf{s} = (j, t, \{d_i\}_{i=1}^N, \{e_i\}_{i=1}^N, \bar{d}, \bar{e}), \quad (3.3)$$

where:

- $j \in \{0, 1\}$ is the jammer state (0 for idle, 1 for active),
- $t \in \{0, \dots, N-1\}$ is the TDMA time slot, determining the primary agent,
- $\{d_i\}_{i=1}^N$ are the data queue states of all agents, where $d_i \in \{0, \dots, D\}$ is the number of packets in agent i 's data queue, and D is the data queue capacity,
- $\{e_i\}_{i=1}^N$ are the energy queue states of all agents, where $e_i \in \{0, \dots, E\}$ is the number of energy units in agent i 's energy queue, and E is the energy queue capacity,
- $\bar{d} = \frac{1}{N} \sum_{i=1}^N \frac{d_i}{D} \in [0, 1]$ is the normalized average data queue state across all agents, providing a summary of the system's data load for cooperative decision-making.
- $\bar{e} = \frac{1}{N} \sum_{i=1}^N \frac{e_i}{E} \in [0, 1]$ is the normalized average energy queue state across all agents, providing a summary of the system's energy load for cooperative decision-making.

For N agents, the state vector $\mathbf{s}$ has $1 + 1 + 2 \times N + 2 = 2N + 4$ components. This continuous state representation is suitable for DQN, which uses a deep neural network to approximate the Q-function over a continuous state space.

In contrast, for the baseline Q-learning algorithm, the state space is discretized to construct a Q-table. The state for agent i is defined similarly where all continuous components are discretized into 6 bins like average data and average energy. The discretized state for agent i is thus:

$$s_i = (j, t, d_i, e_i, \text{bin}(\bar{d}), \text{bin}(\bar{e})), \quad (3.4)$$

where $\text{bin}(\cdot)$ denotes the discretization function. The total number of discrete states per agent is:

$$|\mathcal{S}_i| = 2 \times N \times (D + 1) \times (E + 1) \times 6 \times 6. \quad (3.5)$$

This allows the Q-learning algorithm to maintain a manageable Q-table for learning.

3.2.2. Action space

The action space for each agent is determined by the system state and the TDMA schedule at time slot t . All agents have the same set of possible actions, but the TDMA schedule prioritizes the primary agent ($i = t \bmod N$) to transmit, with non-primary agents ($i \neq t \bmod N$) potentially transmitting only if the primary agent does not.

The transmitter has an action space containing $(M + 4)$ possible actions. These actions include staying idle, actively transmitting data, harvesting energy from jamming signals, backscattering jamming signals, and selecting one of M transmission rates using a rate adaptation (RA) technique when the system is under jamming. Thus, the action space is defined as:

$$\mathcal{A} = \{a \mid a \in \{1, \dots, M + 4\}\}, \quad (3.6)$$

Where:

$$a = \begin{cases} 1, & \text{stay idle,} \\ 2, & \text{actively transmit data,} \\ 3, & \text{harvest energy from the jamming signals,} \\ 4, & \text{backscatter data using the jamming signals,} \\ 4 + m, & \text{adapt the transmission rate to} \\ & \text{rate } r_m \text{ with } m \in \{1, \dots, M\}. \end{cases}$$

The decision-making process of the transmitter for primary agent is illustrated in Figure 3.2. Notably, the transmitter may always opt to remain idle. When the jammer is inactive, and provided there is data available in the queue and sufficient energy is stored, the transmitter can actively send data to the receiver. Conversely, during jamming

attacks, it has the option to harvest energy from the interference. If data is available in the queue, the transmitter may also choose to backscatter the data using the jamming signals. Furthermore, if the energy reserve is sufficient, the transmitter can adopt the RA technique to adjust its transmission rate and send data to the receiver.

The TDMA schedule enforces that only one agent performs a transmission action ($a \in \{2, 4, 5, \dots, 4 + M\}$) per time slot:

- **Primary Agent** ($i = t \bmod N$): Evaluated first and can select any action from $\mathcal{A}_i$, including transmission actions if available. If the primary agent selects a transmission action, it transmits, and non-primary agents are assigned non-transmission actions ($a \in \{1, 3\}$).
- **Non-Primary Agents** ($i \neq t \bmod N$): If the primary agent does not select a transmission action, non-primary agents are evaluated for transmission actions. The agent with the highest Q-value for a transmission action (or a random one during exploration) is selected to transmit, while all other agents (including the primary agent) are assigned non-transmission actions ($a \in \{1, 3\}$).

The process can be describe in Figure 3.3. The joint action space for all agents at time slot t is the product of individual action spaces, constrained by the TDMA schedule such that exactly one agent performs a transmission action (if any), and all others perform non-transmission actions. This ensures that the system operates on a single channel ($C = 1$), avoiding collisions.

3.2.3. Immediate reward

In this study, we consider a multi-agent system where each agent i (for $i = 1, 2, \dots, N$, with N being the number of agents) operates in a shared state s . The immediate reward for each agent i is defined as the number of packets that agent i successfully delivers to the receiver, given the joint action $a = (a_1, a_2, \dots, a_N)$, where a_i is the action taken by agent i . The joint state s captures the collective environment, including the states of all agents and the communication channel. The immediate reward function for agent i , denoted $r_i(s, a)$, is thus determined based on the successful packet deliveries attributed to agent i , while considering the impact of the joint action a on the shared environment.

The reward function $r_i(s, a)$ for each agent i is given by:

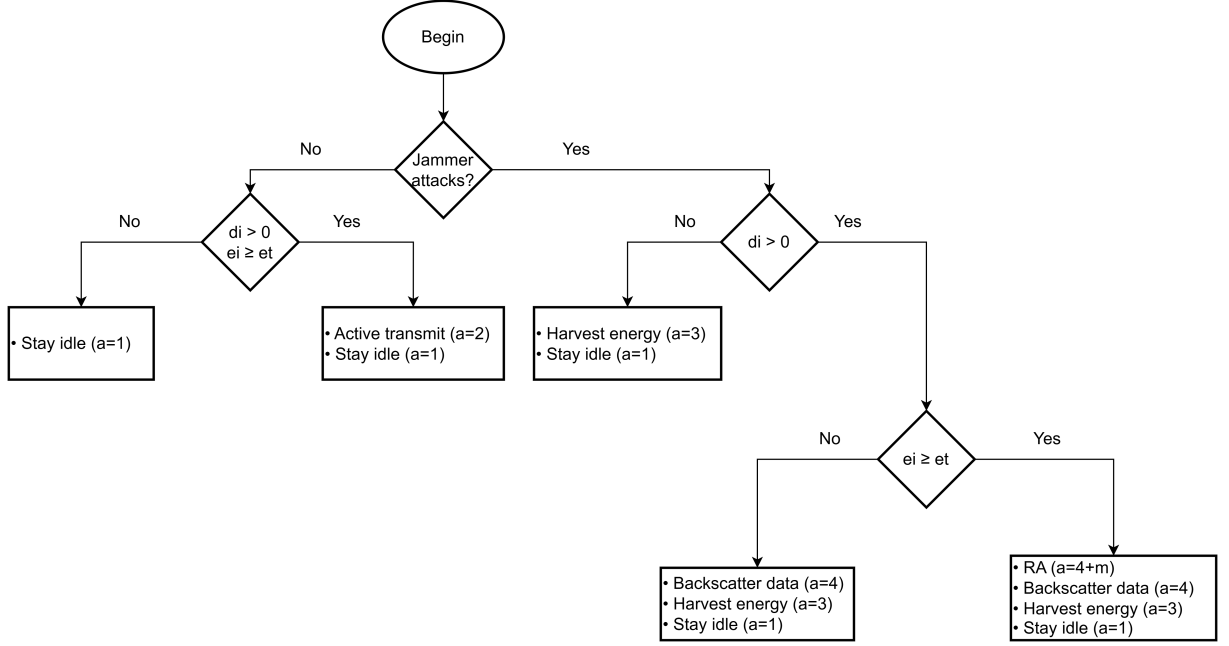


Figure 3.2. Flowchart to express the possible actions of the transmitter.

$$r_i = \begin{cases} d_t(i), & (j = 0, a_i = 2, d_i > 0, e_i \geq e_t, 0 < d_t(i) \leq \hat{d}_t) \\ d_n^J(i), & (j = 1, a_i = 4, d_i > 0, 0 < d_n^J(i) \leq \hat{d}_n^J) \\ d_m^r(i), & (j = 1, a_i = 4 + m, d_i > 0, e_i \geq e_t, 0 < d_m^r(i) \leq \hat{d}_m^r) \\ 0, & \text{otherwise,} \end{cases} \quad (3.7)$$

where:

- $d_t(i)$ is the number of packets actively transmitted by agent i (up to $\hat{d}_t$),
- $d_n^J(i)$ is the number of packets backscattered by agent i (up to $\hat{d}_n^J$), with losses if $d_n^J(i) > \hat{d}_n^J$,
- $d_m^r(i)$ is the number of packets transmitted by agent i via rate adaptation (up to $\hat{d}_m^r$), with losses if the selected rate exceeds the channel capacity under jamming,
- The immediate reward is 0 if the transmitter fails to successfully send any packet to the receiver.

In this multi-agent backscatter communication system, each agent i learns its own policy π_i to maximize its individual long-term average throughput, based on its individual reward. The agents operate in a shared environment with joint state s_k , and their

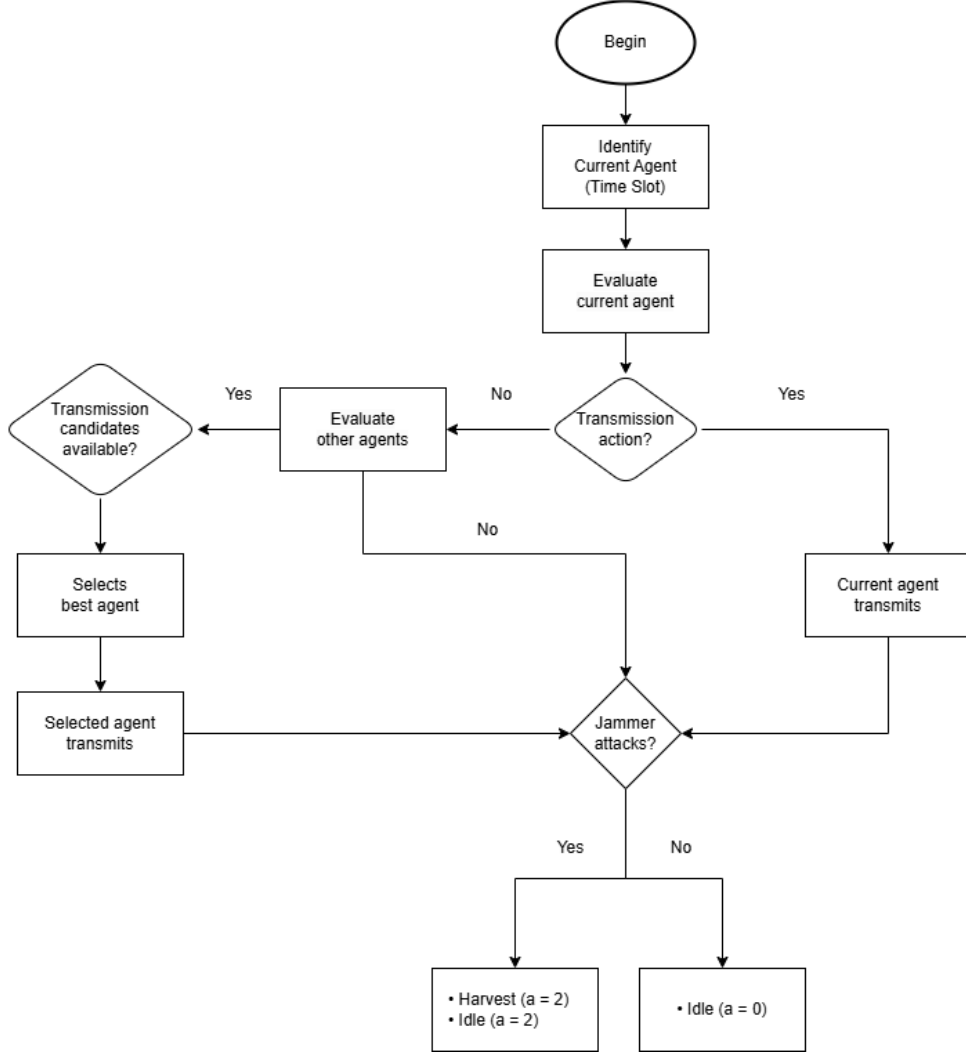


Figure 3.3. Flowchart of Transmitter Decision Process in Multi-Agent System.

actions $a_k = (a_{1,k}, a_{2,k}, \dots, a_{N,k})$ are determined by the joint policy $\pi = (\pi_1, \pi_2, \dots, \pi_N)$. The optimization objective for each agent i is to find a policy π_i^* that maximizes its own throughput, while the cooperative goal of maximizing total system throughput emerges from the collective behavior of the agents, facilitated by coordination mechanisms like TDMA and shared environmental constraints such as jamming.

The optimization objective for agent i is:

$$\max_{\pi_i} \mathcal{R}_i(\pi_i, \pi_{-i}) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{k=1}^T \mathbb{E} [r_{i,k}(s_k, a_k)], \quad (3.8)$$

where π_i is the policy of agent i , $\pi_{-i} = (\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_N)$ represents the policies of all other agents, $a_k = (a_{1,k}, \dots, a_{N,k})$ with $a_{i,k} = \pi_i(s_k)$, and $r_{i,k}(s_k, a_k)$ is the individual reward for agent i at time slot k . It is important to note that the Markov Decision

Process (MDP) defined is irreducible, meaning that from any given state, the process can eventually reach any other state in a finite number of steps, and there are no terminal states.

3.3. Cooperative Multi-Agent Q-Learning and DQN for Anti-Jamming

3.3.1. Applying Cooperative Multi-Agent Q-Learning

In our backscatter communication network, we apply cooperative multi-agent Q-learning to develop an anti-jamming strategy that enables multiple users to optimize their actions in a shared TDMA environment. Each user (agent) operates in a time slot, deciding whether to actively transmit, backscatter, harvest energy, or adapt transmission rates based on the jammer state, their data queue, and energy levels. The state space for each agent includes the jammer state $j \in \{0, 1\}$, the current time slot $t \in \{0, \dots, N - 1\}$ (where N is the number of users), the agent's data queue d , energy level e , and the average data and energy queue sizes of all users. To manage the large state space, we discretize continuous values into bins (e.g., 0 to 5 based on thresholds 1/6 each). The action space for each agent is dynamic. When the jammer is inactive ($j = 0$), an agent can actively transmit (action 1) if it has data and sufficient energy ($e \geq e_t$). The reward for each action reflects the number of packets successfully transmitted or energy harvested, with penalties for packet loss due to jamming or insufficient rate adaptation. In cooperative multi-agent Q-learning, each agent maintains its own Q-table, $Q(s, a)$, which is updated iteratively using the Q-learning update rule:

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a_{t+1}} Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t) \right], \quad (3.9)$$

where α is the learning rate, γ is the discount factor, r_t is the reward of the agent, and s_{t+1} is the next state.

The agents use an ϵ -greedy strategy for action selection, balancing exploration and exploitation. Cooperation is achieved through the TDMA structure, where only one agent transmits per time slot, while others harvest or remain idle. The optimal policy $\pi^*(s) = \arg \max_a Q^*(s, a)$ maximizes the throughput while minimizing packet loss due to jamming. However, traditional Q-learning struggles with scalability in our scenario due to the large state and action spaces caused by multiple users and dynamic jammer states. The discretized state space, while manageable for small N , grows exponentially as N increases, leading to slow convergence and high memory requirements. This limi-

tation motivates the use of Deep Q-Learning (DQN) to approximate the Q-function, as discussed in the next subsection.

Algorithm 3.1 Cooperative Multi-Agent Q-Learning (Core Steps)

- 1: **Initialize:** $Q^i(s, a) = 0$ for each agent i , $\alpha, \gamma, \epsilon = 1.0$
 - 2: **for** $t = 1$ to T **do**
 - 3: Check which Agent of current timeslot $i \leftarrow t \bmod N$
 - 4: Get state s_i
 - 5: Choose a^i using ϵ -greedy: $a^i \leftarrow \arg \max_a Q^i(s, a)$ or random
 - 6: Cooperate: Select one agent to transmit based on Q -values - prioritize the primary agent, others harvest/idle
 - 7: Get rewards and states $(r, s') \leftarrow \text{env.step}(a)$
 - 8: Update: $Q^i(s, a^i) \leftarrow Q^i(s, a^i) + \alpha [r_i + \gamma \max_{a'} Q^i(s', a') - Q^i(s, a^i)]$
 - 9: **end for**
-

3.3.2. Applying Cooperative Multi-Agent DQN

To address the scalability issues of Q-learning, we propose a cooperative multi-agent Deep Q-Network (DQN) approach for anti-jamming in our backscatter communication network. Each agent is equipped with its own DQN consisting of five fully connected layers. The network uses the hyperbolic tangent ($\tanh$) activation function for hidden layers to ensure stable training. The state representation is the same as in Q-learning but remains continuous, including the states of all agents rather than just its own, allowing the DQN to generalize across states. The DQN framework incorporates several key mechanisms to enhance learning stability and efficiency:

- **Experience Replay:** Each agent stores its experiences (s_t, a_t, r_t, s_{t+1}) in a replay memory, a deque with a maximum length of `memory_size`. During training, a random mini-batch of size `batch_size` is sampled from this memory for learning. This approach reduces correlation between consecutive experiences and improves sample efficiency.
- **Target Network:** Each agent maintains a target network (`target_models`) alongside the primary Q-network (`models`). The target network, with parameters θ' , is updated periodically (every N time slots) by copying the primary network's parameters θ , using `update_target`. This reduces oscillations in Q-value estimates and stabilizes training.

- **Loss Function and Optimization:** The Q-network is trained by minimizing the Smooth L1 Loss (Huber loss) between the predicted Q-values and the target Q-values, as implemented in the `replay` method. The target Q-value for user i is computed as $r + \gamma \max_{a'} \hat{Q}(s', a'; \theta')$, where $\hat{Q}$ is the target network. The network parameters are updated using the Adam optimizer with a learning rate `learning_rate_deepQ`.

The training process operates over T time slots. In each time slot, the current agent (determined by the TDMA schedule) selects an action using an ϵ -greedy policy, with ϵ decaying from 1.0 to `epsilon_min` at a rate of `epsilon_decay`. If the current agent chooses to transmit, others harvest energy if the jammer is active; otherwise, a candidate agent with the highest Q-value may transmit, ensuring cooperative behavior. The environment evolves based on the `step` method, updating jammer states, data queues, and energy levels, while tracking metrics like packet loss ratios and average rewards. The cooperative multi-agent DQN approach significantly improves scalability and convergence speed compared to Q-learning, as the neural network approximates the Q-function without requiring an explicit Q-table. By leveraging shared state information and coordinated actions via TDMA, the agents collectively learn an optimal anti-jamming strategy, maximizing throughput and minimizing packet loss in the presence of a probabilistic jammer.

Algorithm 3.2 Cooperative Multi-Agent DQN (Core Steps)

- 1: **Initialize:** $Q^i(s, a; \theta^i)$, $\hat{Q}^i(s, a; \theta^i)$ for each agent i , replay memory D , $\epsilon = 1.0$
 - 2: **for** $t = 1$ to T **do**
 - 3: Agent $i \leftarrow t \bmod N$
 - 4: Get state $s \leftarrow (j, t, \{d^i\}, \{e^i\}, \text{avg}(d), \text{avg}(e))$
 - 5: Choose a^i using ϵ -greedy: $a^i \leftarrow \arg \max_a Q^i(s, a; \theta^i)$ or random
 - 6: Cooperate: Select one agent to transmit - prioritize the primary agent, others harvest/idle
 - 7: Get rewards and states $(r, s') \leftarrow \text{env.step}(a)$
 - 8: Store $(s, a, r, s', \{r^i\})$ in D
 - 9: Sample mini-batch from D
 - 10: $y^i \leftarrow r_i + \gamma \max_{a'} \hat{Q}^i(s', a'; \theta^i)$
 - 11: Update θ^i by minimizing $\text{SmoothL1Loss}(Q^i(s, a^i; \theta^i), y^i)$
 - 12: **if** $t \bmod C = 0$ **then**
 - 13: $\theta^i \leftarrow \theta^i$
 - 14: **end if**
 - 15: **end for**
-

CHAPTER 4

SIMULATION SETUP AND PERFORMANCE EVALUATION

4.1. Experimental Setup Parameters

In this section, we detail the simulation setup and parameter configurations for evaluating the performance of the proposed cooperative multi-agent deep reinforcement learning (DRL) anti-jamming method in backscatter communication networks. The system parameters define the operational constraints and dynamics of the multi-agent system, while the algorithm parameters specify the configurations for the Q-learning and DQN algorithms. The performance of the proposed DQN-based method is compared against a Q-learning baseline and a greedy anti-jamming strategy that does not employ reinforcement learning.

4.1.1. System Parameters

The system under consideration consists of N numbers of agents (transmitters), a receiver, and a jammer, operating in a backscatter communication network with a single channel. The agents communicate using a Time Division Multiple Access (TDMA) scheduling mechanism, where each time slot is assigned to a primary agent (i.e., agent $i = t \bmod N$ at time slot t). The key system parameters are as follows:

Each agent maintains a data queue of capacity $D = 10$ packets and an energy queue of capacity $E = 10$ units. Data packets arrive at each agent's data queue according to a Poisson distribution with an average arrival rate of $\lambda = 3$ packets per time slot. If the data queue is full, arriving packets are dropped.

When the jammer is idle ($j = 0$), the primary agent can actively transmit up to $\hat{d}_t = 4$ packets to the receiver, consuming $e_t = 1$ unit of energy per packet, provided there are sufficient data and energy in its queues. When the jammer is active ($j = 1$), the primary agent can either harvest energy from the jamming signals, backscatter data, or perform rate adaptation. The jammer operates in two states: idle ($j = 0$) or active ($j = 1$), with a probability of being idle $\nu = 0.1$ and being active $\nu = 0.9$.

When active, the jammer selects a power level from a discrete set with probabilities $\mathbf{x} = [0.6, 0.2, 0.2]$, corresponding to three power levels. The harvested energy per

jamming level is $e_n^J \in \{1, 2, 3\}$ units, and the number of packets that can be successfully backscattered per jamming level is $\tilde{d}_n^J \in \{1, 2, 3\}$, both selected according to the probabilities $\mathbf{x}$. The fixed backscatter rate is $b^\dagger = 3$ packets; if $b^\dagger > \tilde{d}_n^J$, the excess packets are lost. For rate adaptation, the primary agent can transmit at rates $\hat{d}_m^r \in \{2, 1, 0\}$ packets per jamming level, consuming $e_t = 1$ unit of energy per packet, with losses if the selected rate exceeds the channel capacity under jamming.

Non-primary agents can either remain idle or harvest energy when the jammer is active, contributing to the system's energy reserves for future transmissions. The total number of possible actions per agent is 7: idle ($a = 1$), active transmission ($a = 2$), energy harvesting ($a = 3$), backscatter ($a = 4$), and three rate adaptation actions ($a = 5, 6, 7$).

4.1.2. Q-learning and DQN Algorithm Parameters

This subsection outlines the parameter settings for the Q-learning and DQN algorithms used in the simulations to determine the optimal anti-jamming policy for the multi-agent system.

4.1.2.1. Q-learning Algorithm

To establish a baseline for comparison, we implement a Q-learning algorithm for the multi-agent system. Each agent maintains its own Q-table, with the state space discretized to manage computational complexity. The total number of discrete states per agent is calculated as:

$$|\mathcal{S}_i| = 2 \times N \times (D + 1) \times (E + 1) \times 6 \times 6, \quad (4.1)$$

where N number of agents.

The Q-learning parameters are set as follows:

- Learning rate: $\alpha = 0.1$
- Discount factor: $\gamma = 0.9$
- Exploration strategy: ϵ -greedy with initial $\epsilon = 1$, minimum $\epsilon_{\min} = 0.01$, and decay rate $\epsilon_{\text{decay}} = 0.9999$
- Number of training iterations: $T = 1,000,000$

The Q-table is updated using the standard Q-learning update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r_i + \gamma \max_{a'} Q(s', a') - Q(s, a) \right), \quad (4.2)$$

where s is the current discrete state, a is the action taken, r_i is the reward, s' is the next state, and $\max_{a'} Q(s', a')$ is the maximum Q-value for the next state over all possible actions.

4.1.2.2. DQN Algorithm

The proposed method employs a deep Q-learning (DQN) algorithm to handle the continuous state space and enable cooperative learning among agents. Each agent has its own Q-network and target network, with identical architectures. The Q-network architecture consists of four fully connected hidden layers with 1024, 512, 256, and 64 neurons, respectively, using the $\tanh$ activation function for each hidden layer. The output layer has 7 neurons, corresponding to the action space size, with a linear activation function to produce Q-values for each action.

The state space for DQN is continuous, with $1 + 1 + 2 \times N + 2$ features for N agents: the jammer state, TDMA time slot, data and energy queue states for all agents, and the normalized average data queue and energy queue state across all agents.

The DQN parameters are configured as follows:

- Learning rate: $\alpha = 0.0001$
- Discount factor: $\gamma = 0.99$
- Exploration strategy: ϵ -greedy with initial $\epsilon = 1$, minimum $\epsilon_{\min} = 0.01$, and decay rate $\epsilon_{\text{decay}} = 0.9999$
- Replay memory size: $D = 50,000$
- Mini-batch size: 64 samples per training iteration
- Number of training iterations: $T = 1,000,000$
- Target network update frequency: Every 2,000 iterations
- Loss function: Smooth L1 loss

The DQN algorithm uses experience replay to sample mini-batches from the replay memory, updating the Q-network parameters via backpropagation. The target network is updated periodically to stabilize training, and the ϵ -greedy strategy balances exploration and exploitation during action selection.

The Q-network architecture comprises four fully connected hidden layers with 1024, 512, 256, and 64 neurons, respectively, followed by an output layer with 7 neurons. This deep structure was selected to capture the complexity of the state-action space and the non-linear relationships inherent in the anti-jamming problem.

- **Number of Layers (4):** Four layers were chosen to balance model capacity and computational efficiency. A deeper network (e.g., 5+ layers) could overfit to the training data or increase training time excessively, given the limited size of the replay memory ($D = 50,000$) and the number of training iterations ($T = 1,000,000$). Conversely, a shallower network (e.g., 2–3 layers) might lack the capacity to model the intricate decision-making required, such as distinguishing between harvesting, backscatter, and active transmission under varying jamming conditions.
- **Neuron Counts (1024, 512, 256, 64):** The decreasing number of neurons across layers ($1024 \rightarrow 512 \rightarrow 256 \rightarrow 64$) implements a funnel-like structure that reduces dimensionality progressively, aiding in feature extraction and preventing information loss. The initial layer with 1024 neurons was chosen to accommodate the high-dimensional input state ($1 + 1 + 2 \times N + 2$ features, e.g., 10 features for $N = 2$), ensuring sufficient capacity to process queue states and jammer dynamics. Subsequent layers (512, 256, 64) refine this representation, with 64 neurons in the final hidden layer providing a compact yet expressive feature set for the output layer.
- **Activation Function (tanh):** The tanh activation function was selected for all hidden layers due to its ability to introduce non-linearity while maintaining output values between -1 and 1, which is suitable for normalizing the continuous state features (e.g., queue levels scaled between 0 and 1). Compared to ReLU, which can lead to dying neuron problems in deep networks, tanh provides smoother gradients, aiding convergence in the presence of the complex reward function.
- **Output Layer (7 Neurons, Linear Activation):** The output layer has 7 neurons, corresponding to the action space size (e.g., idle, harvest, transmit, backscatter, and

rate adaptation actions). A linear activation was used to produce raw Q-values, allowing the network to learn the expected cumulative reward for each action without constraining the output range, which is standard in DQN implementations.

The hyperparameters were tuned based on empirical performance and DRL best practices:

- **Learning rate** ($\alpha = 0.0001$): A low learning rate was chosen to ensure stable convergence, given the deep network’s capacity to overfit.
- **Discount factor** ($\gamma = 0.99$): A high value reflects the long-term nature of the anti-jamming task, where future rewards (e.g., successful transmissions after harvesting) are nearly as important as immediate rewards.
- **ϵ -greedy strategy**: Initial $\epsilon = 1$ ensures full exploration, decaying to $\epsilon_{\min} = 0.01$ with $\epsilon_{\text{decay}} = 0.9999$ to balance exploration and exploitation. This slow decay supports learning in a dynamic jamming environment, where extensive exploration is needed initially.
- **Replay memory size** ($D = 50,000$): A large buffer was selected to store diverse experiences, reducing correlation in mini-batches and stabilizing training.
- **Mini-batch size (64)**: This size balances computational efficiency and gradient estimation accuracy, a common choice in DQN implementations.
- **Training iterations** ($T = 1,000,000$): Extensive iterations ensure convergence, given the multi-agent complexity and the paper’s simulation length.
- **Target network update frequency (every 2,000 iterations)**: This interval stabilizes training by reducing target value oscillations.
- **Loss function (Smooth L1 loss)**: Chosen for its robustness to outliers in Q-value updates, combining the benefits of L1 and L2 losses, and widely used in DQN to handle noisy rewards.

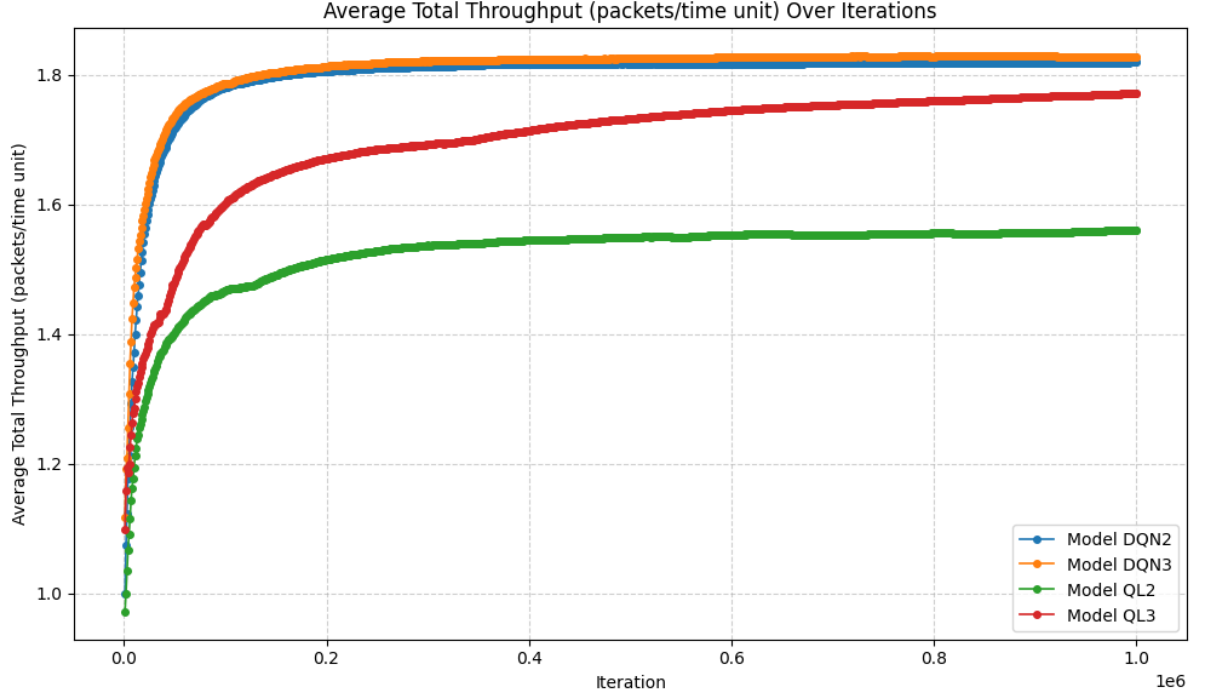


Figure 4.1. Convergence rates of Q-learning and deep Q-learning algorithms by total rewards.

4.2. Simulation Results

4.2.1. Convergence Comparison of the Q-learning and DQN Reinforcement Learning Algorithms

Figure 4.1 shows that the Deep Q-Learning algorithm has a better convergence rate than the Q-Learning algorithm when increasing the number of cooperative agents. The DQN agents achieve significantly higher rewards, around 1.8, compared to Q-Learning agents which take more number of agents to reach the result similar to DQN. This indicates that DQN learns more efficiently, especially in cooperative multi-agent scenarios. Similarly, Figure 4.2 shows the same trend, with the average performance of DQN agents outperforming that of Q-Learning agents.

Figure 4.3 shows the percentage of packet loss caused by both data queue overflow and transmission-related issues, while Figure 4.4 illustrates only the packet loss due to transmission. The latter includes losses resulting from agent strategies such as rate adaptation or backscattering. Both figures demonstrate that DQN achieves better learning performance, as it results in lower packet loss in both cases. Figure 4.4 shows a curve that results from DQN and QL agents learning the optimal strategy to handle jamming as iterations increase (e.g., backscattering when jamming power is high and rate adapta-

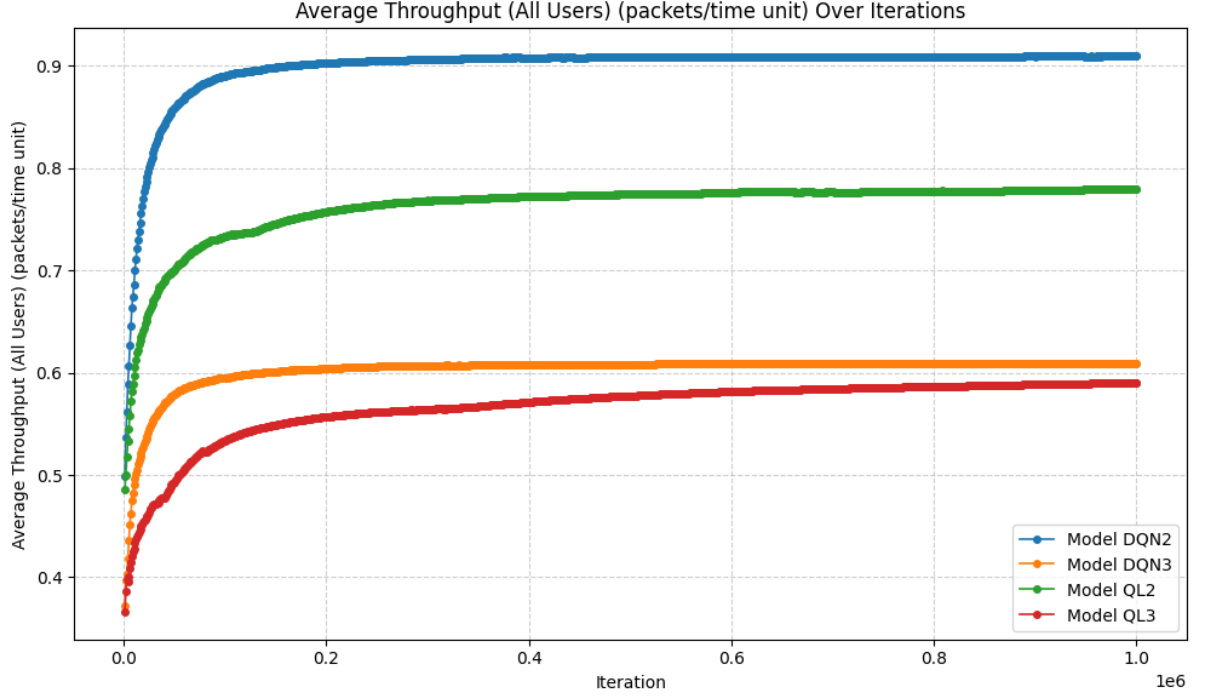


Figure 4.2. Convergence rates of Q-learning and deep Q-learning algorithms by average rewards.

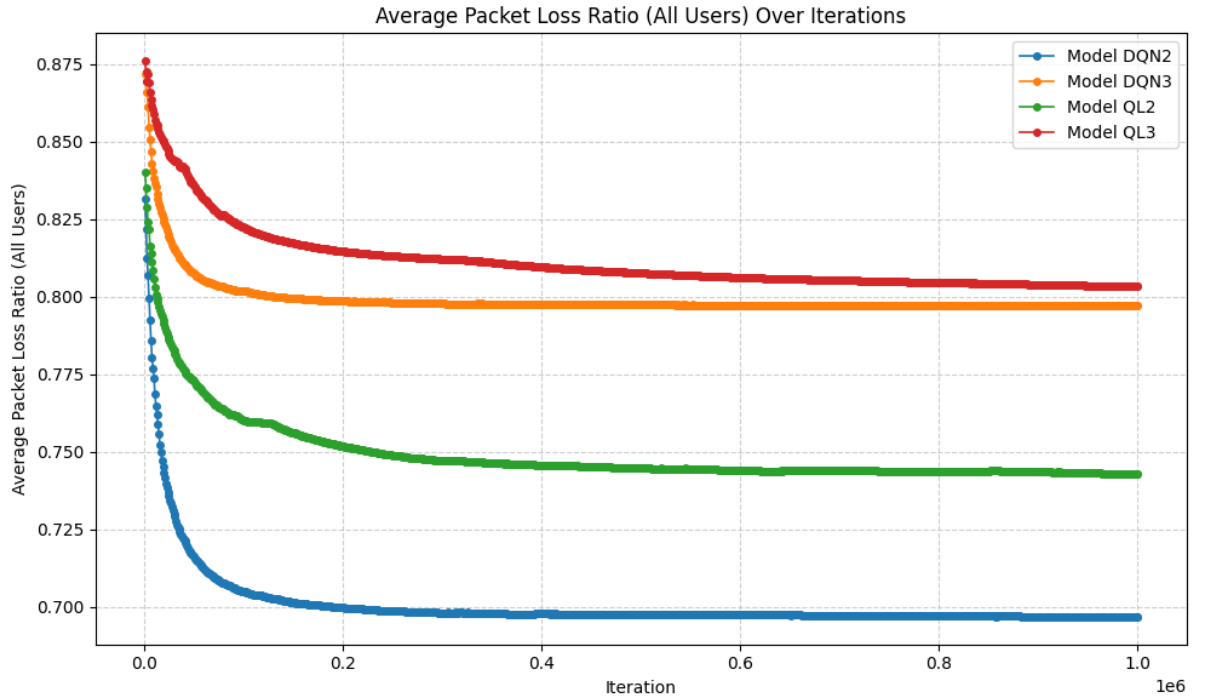


Figure 4.3. Packets loss ratio of Q-learning and deep Q-learning algorithms.

tion when jamming power is low to minimize packet loss). The DQN agents demonstrate more stable learning than the QL agents.

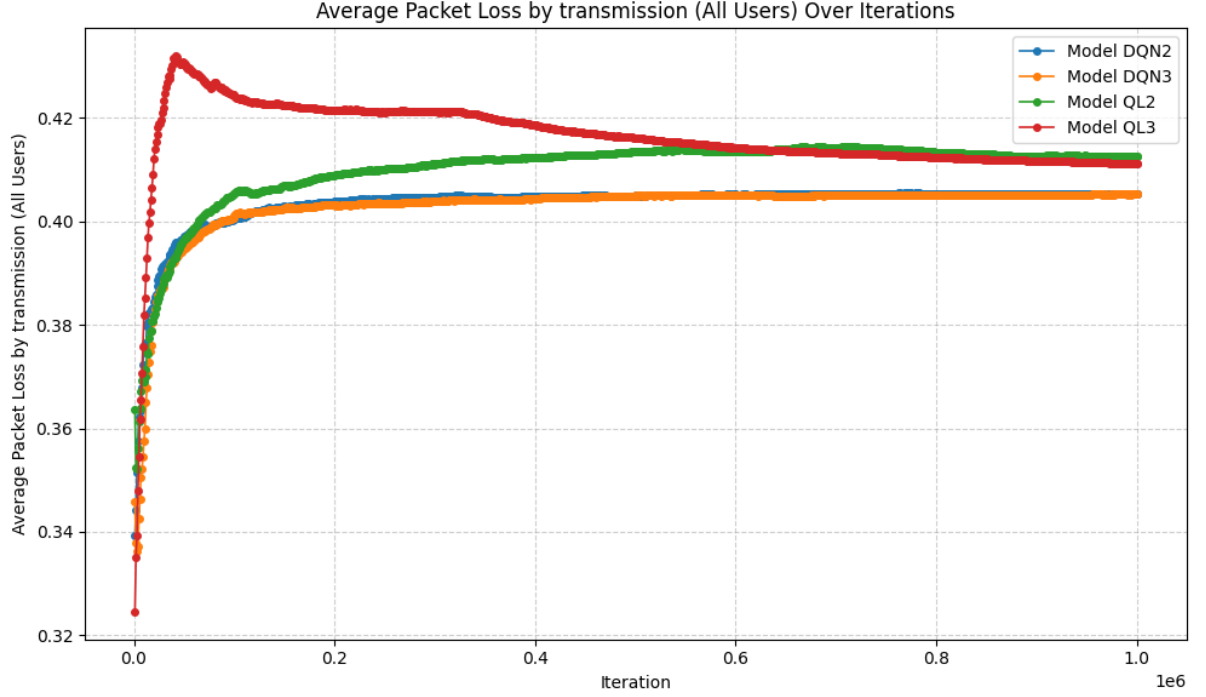


Figure 4.4. Packets loss ratio by transmission of Q-learning and deep Q-learning algorithms.

Additionally, Figure 4.3 and Figure 4.4 complement each other in explaining the learning behavior. Initially, the packet loss ratio by transmission is low because the agents rarely use transmission strategies such as active transmit, backscatter, or rate adaptation. This leads to a high overall packet loss ratio, as incoming data overflows the queue due to the lack of timely transmission. However, as training progresses and the agents begin transmitting more frequently, they learn to adopt optimal strategies—such as using backscatter under high jamming power and rate adaptation under low jamming power. These strategies help minimize packet loss while maximizing successful transmissions. Consequently, the packet loss ratio by transmission stabilizes in the later iterations, as shown in Figure 4.4.

4.2.2. Performance Evaluation Under Different Number of Agent

*DQN Agents

Figure 4.5 illustrates the convergence rate of cooperative agents with varying group sizes over 1,000,000 iterations. The results suggest that increasing the number of agents up to a certain point leads to a faster learning rate in the early stages of training. This may be due to the fact that, while an agent waits for its assigned timeslot, it has time to harvest energy. As a result, strategies such as active transmission or rate adaptation

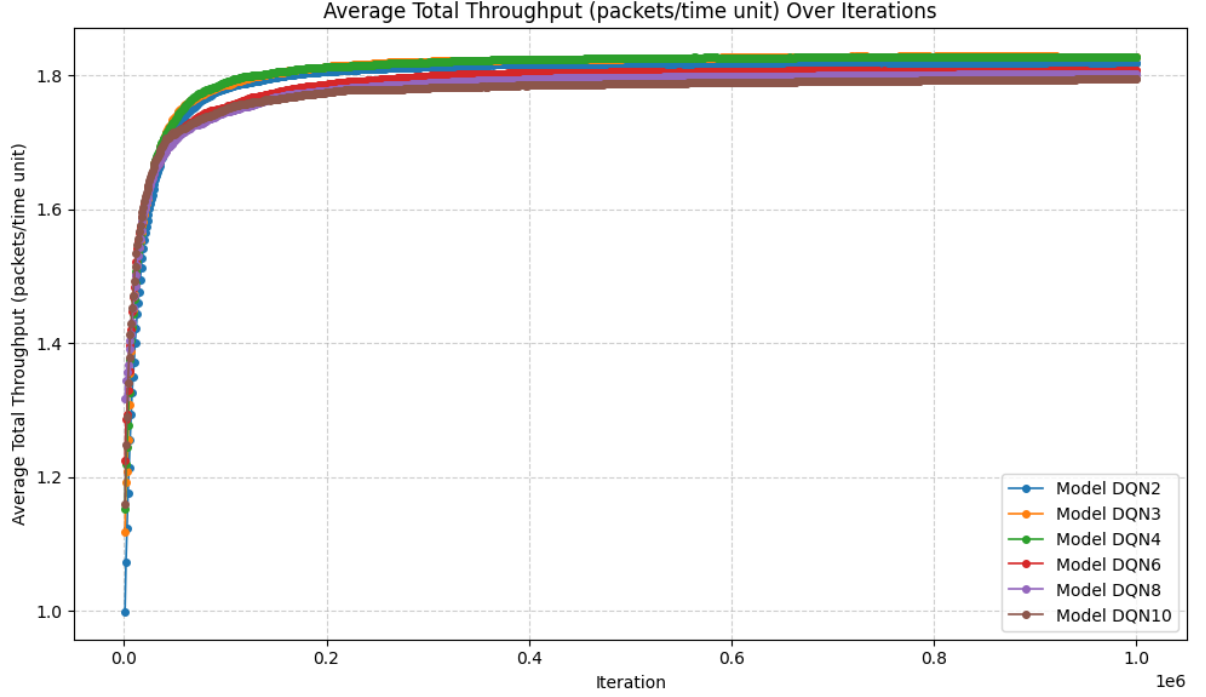


Figure 4.5. Convergence Rates of DQN Agents over 1M Iterations.

become viable when the agent's timeslot arrives. With a larger number of agents, it is more likely that, at any given iteration t , an agent has already harvested sufficient energy to transmit immediately, reducing idle time and accelerating learning.

However, while this advantage is beneficial in the early phase, it may also introduce drawbacks, which will be discussed in a later section. As the number of agents becomes large, DQN learning slows down and requires more iterations to reach convergence. This could be due to the increased number of states and agents, making it more difficult for agents to discover optimal strategies and coordinate effectively.

Figure 4.7 presents the percentage of packet loss caused by both data queue overflow and transmission loss for each group. Based on the graph, groups with more agents tend to experience higher packet loss. This may be due to the drawback mentioned previously: while having more agents can speed up learning initially, it also increases the chance of data queue overflow. Agents must wait for their timeslot or permission to transmit, and with more agents, the waiting time grows—leading to more data accumulating in the queue before transmission is possible.

Figure 4.8 presents the packet loss ratio by transmission for different groups of agents. While the total packet loss appears to increase with a higher number of agents,

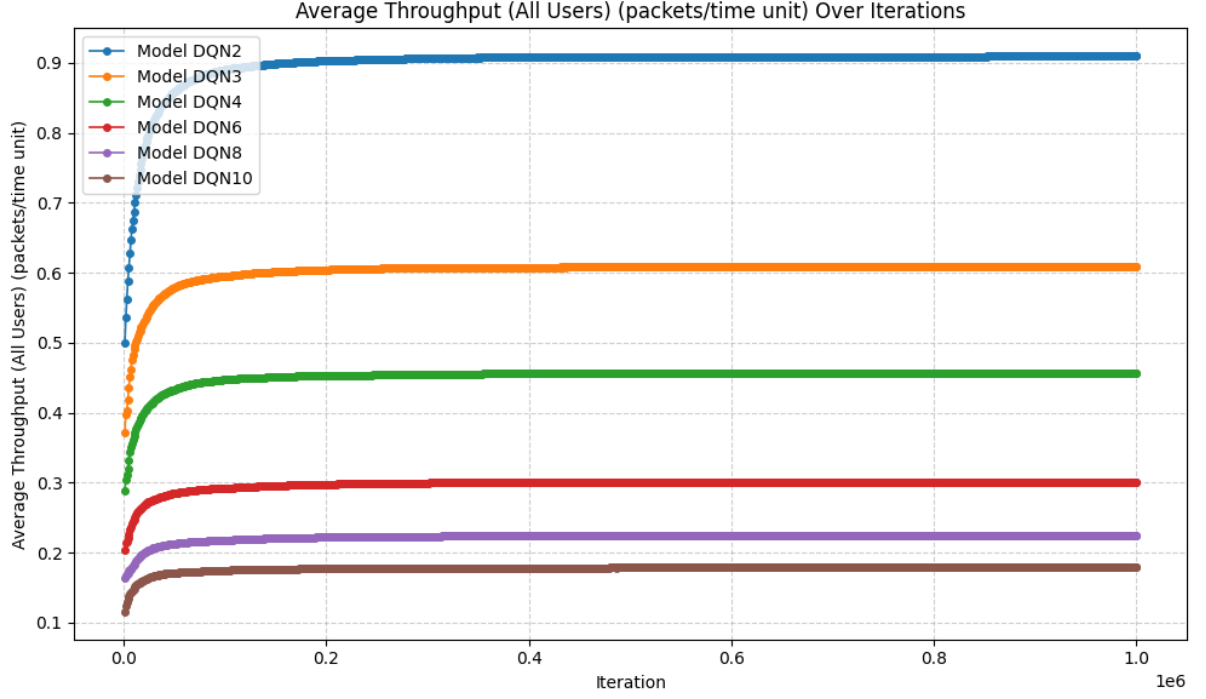


Figure 4.6. Convergence Rates of DQN Agents by average throughput over 1M Iterations.

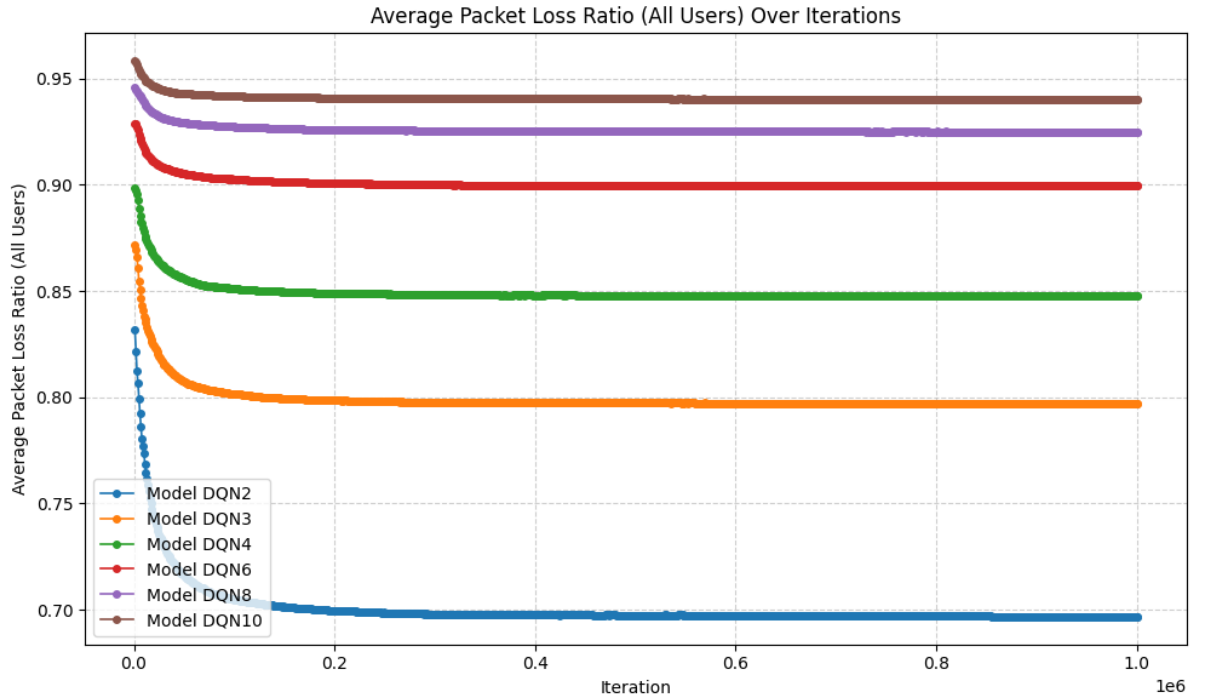


Figure 4.7. Packets loss ratio of DQN agents.

the results suggest that the multi-agent system may learn more effective anti-jamming strategies compared to a single DQN agent. An explanation for this observation is that

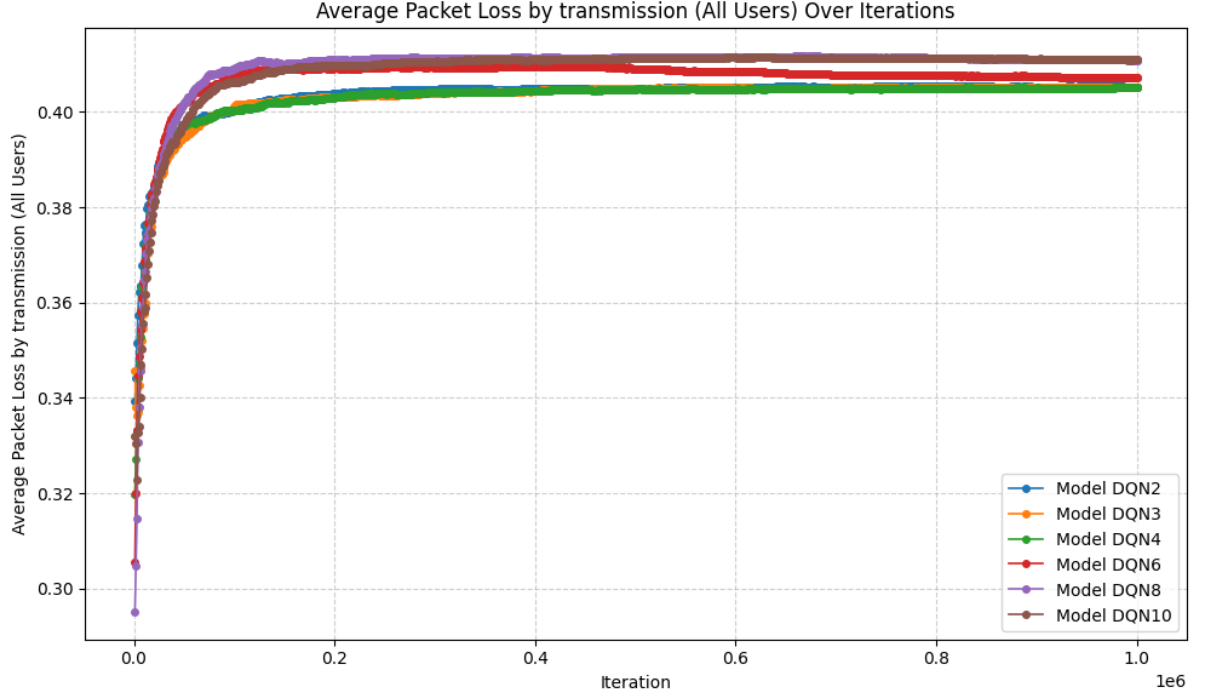


Figure 4.8. Packets loss ratio by transmission of DQN agents.

the collective energy resources available to multiple agents allow them to implement a wider range of or more computationally intensive jamming mitigation techniques.

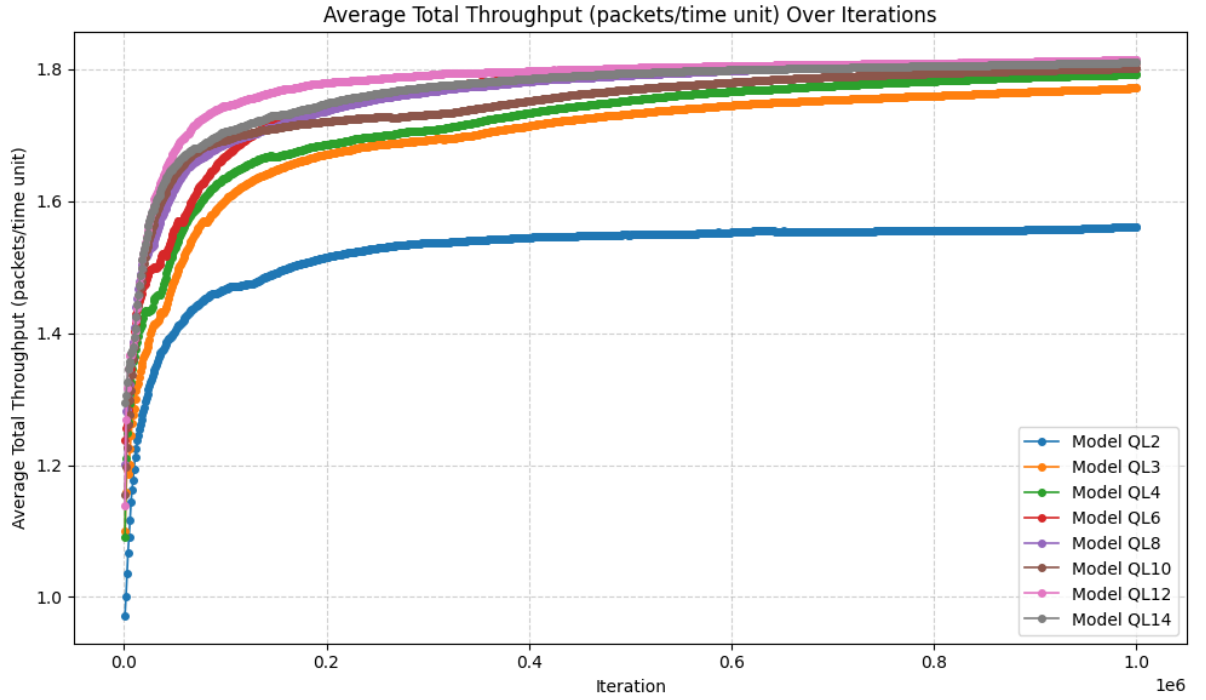
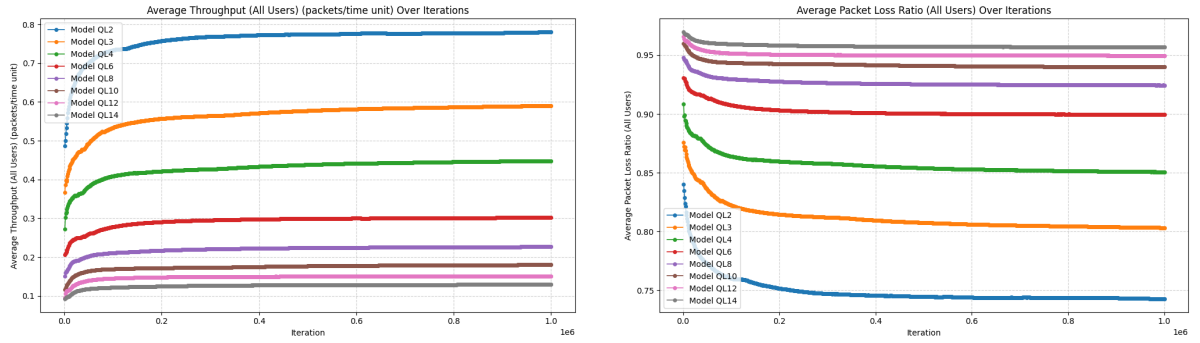


Figure 4.9. Convergence Rates of QL Agents over 1M Iterations.

*QL Agents

Figure 4.9 shows similar trends to the DQN agents, but it requires a larger number of agents to achieve results comparable to DQN. Additionally, beyond a certain number of agents, Q-Learning becomes slower to converge and requires more iterations. The figure also demonstrates that QL agents exhibit less stable learning behavior. This is further supported by Figure 4.11, which shows that QL agents experience more fluctuation in packet loss rates during transmission, indicating less effective learning of optimal strategies compared to DQN.



(a) Convergence Rates of QL Agents by average throughput.

(b) Packets loss ratio of QL agents.

Figure 4.10. (a) Convergence Rates of QL Agents by average, (b) Packets loss ratio of QL agents.

Figure 4.10 also shows the same trend as the DQN agents as the number of agents increases.

4.2.3. Comparison of Different Learning Methods

* Multi-Agent System vs. Single-Agent System

This section provides a comparative analysis of multi-agent and single-agent systems in terms of convergence behavior, throughput, and packet loss performance. The goal is to evaluate how agent collaboration in a multi-agent setup impacts overall network efficiency, learning stability, and adaptability in dynamic environments, compared to an isolated single-agent approach.

Figure 4.12 shows the convergence rates of multi-agent vs. single-agent systems for both DQN and Q-learning (QL) agents. The multi-agent system demonstrates significantly better performance, achieving approximately 1.8 packets/time unit for DQN and 1.5 packets/time unit for QL. In contrast, the single-agent system achieves only around

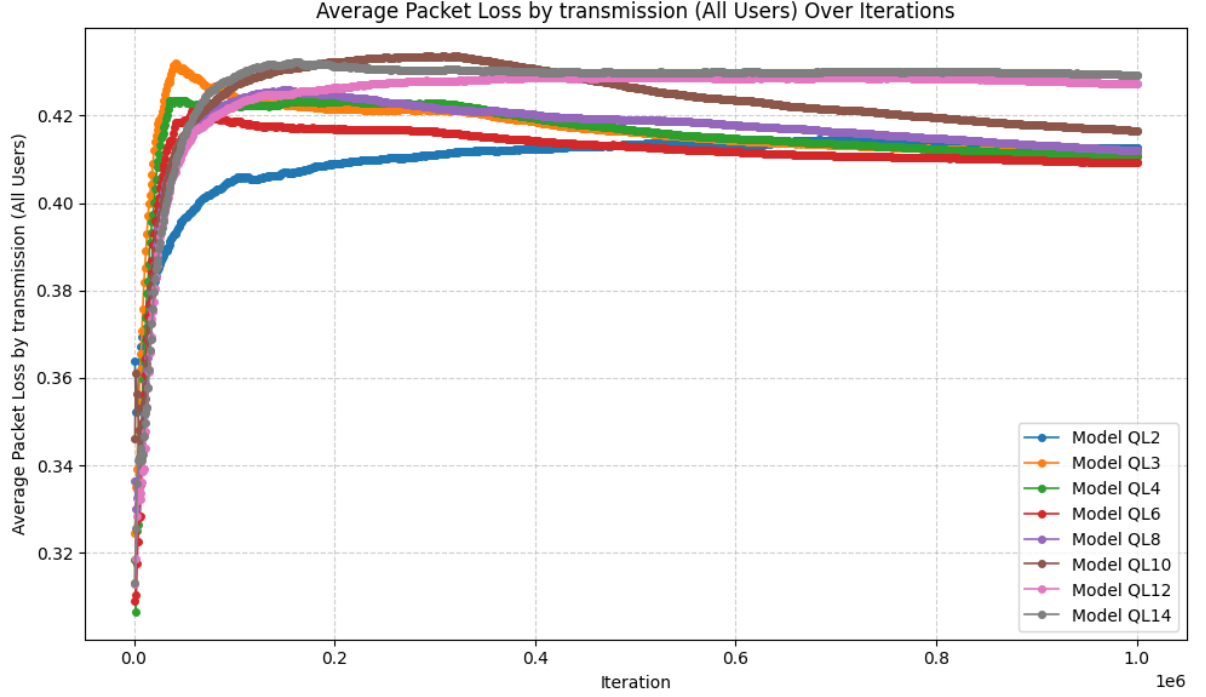


Figure 4.11. Packets loss ratio by transmission of QL agents.

1.4 packets/time unit for DQN and 1.35 packets/time unit for QL. This highlights the better efficiency of the multi-agent system, as it leverages coordination between agents, allowing one agent to transmit while another harvests energy. This ensures that agents have sufficient energy to select optimal strategies during their respective time slots. In comparison, the single-agent system must choose between harvesting and transmitting, which slows down throughput, as illustrated in the figure.

Despite the improved throughput, the multi-agent system also exhibits certain drawbacks, as previously discussed. Figure 4.13 shows that the packet loss ratio is higher in the multi-agent system—approximately 0.7 for DQN and 0.8 for QL—compared to about 0.53 for DQN and 0.55 for QL in the single-agent system. This increased packet loss is due to the fact that, in the multi-agent setting, each agent must wait for its allocated timeslot to transmit. However, data continues to arrive at each agent over time. As a result, the data queue can overflow while the agent is waiting for its turn, leading to dropped packets. This highlights a trade-off between improved coordination (and throughput) and the challenge of managing queue stability in time-slotted multi-agent systems.

While the multi-agent system has a higher overall packet loss ratio, it shows a lower

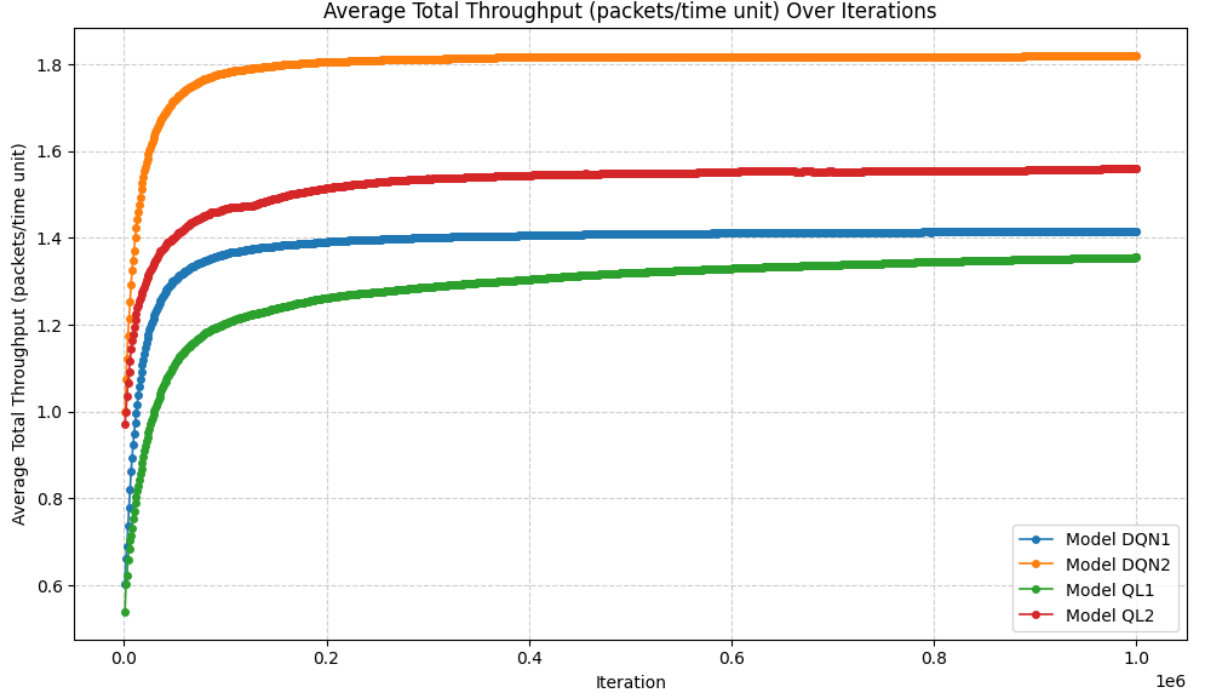


Figure 4.12. Convergence rates of Multi-Agent vs. Single-Agent systems.

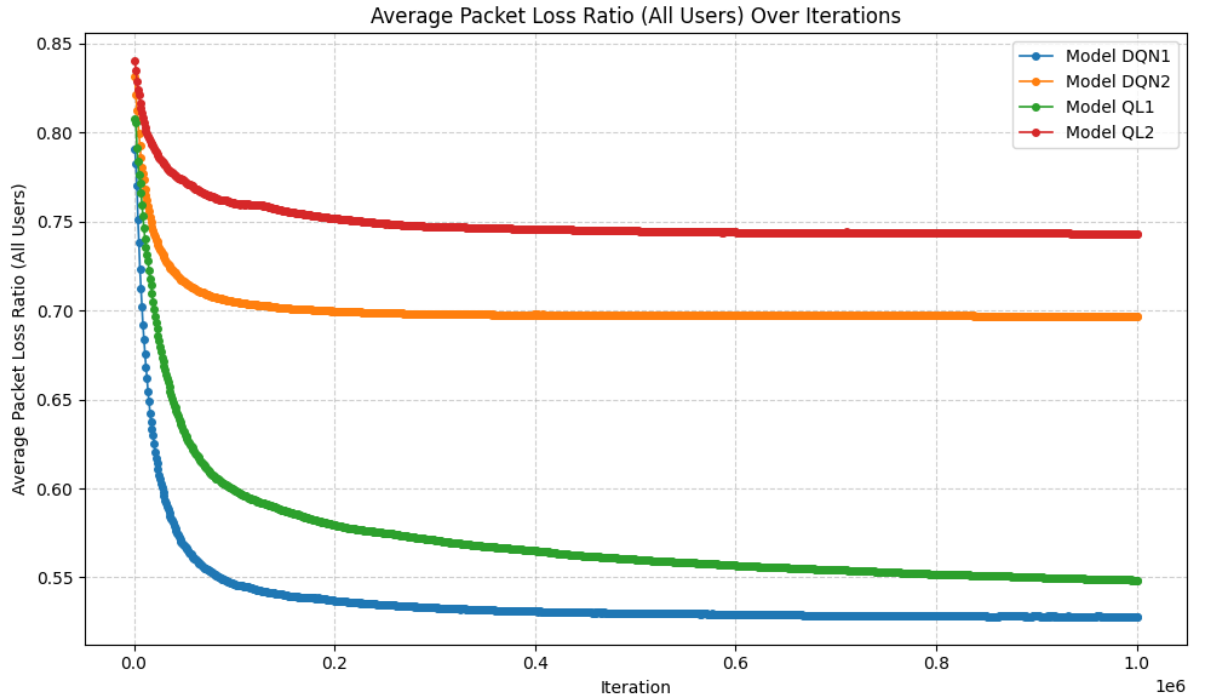


Figure 4.13. Packets loss ratio of Multi-Agent vs. Single-Agent systems.

packet loss ratio by transmission, as illustrated in Figure 4.14. This indicates that the multi-agent system learns more effective transmission strategies than the single-agent system, resulting in fewer packet losses during actual transmissions.

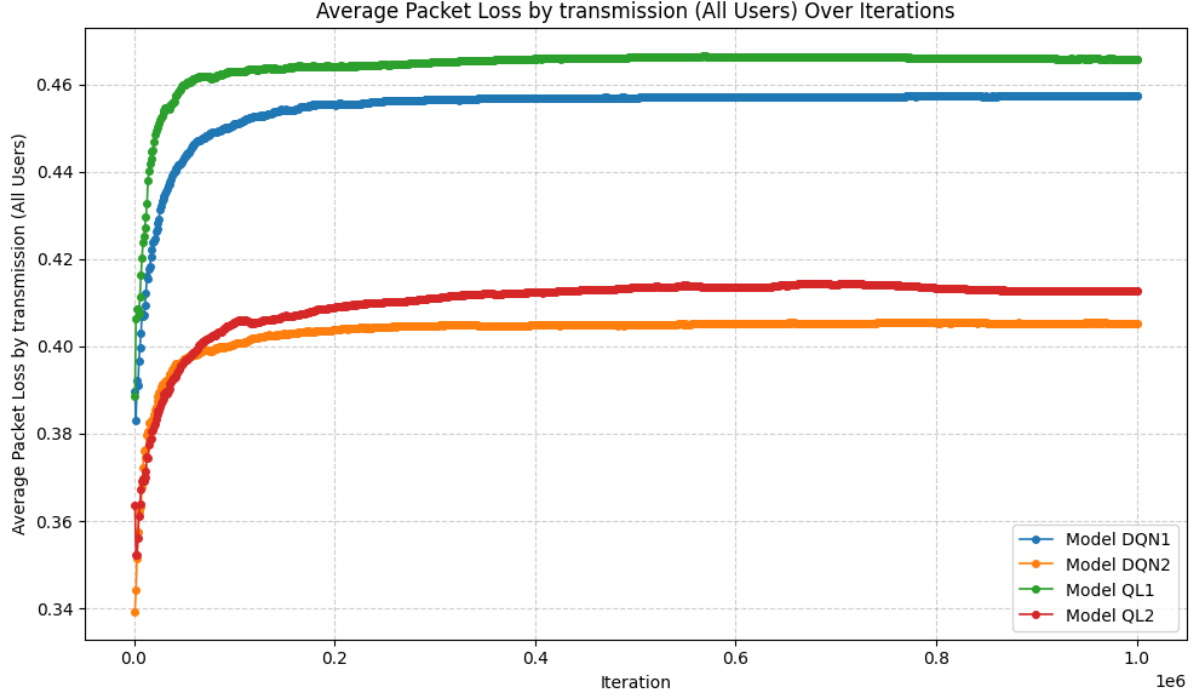


Figure 4.14. Packets loss ratio by transmission in Multi-Agent vs. Single-Agent systems.

* Proposed Solution vs. HTT Scheme

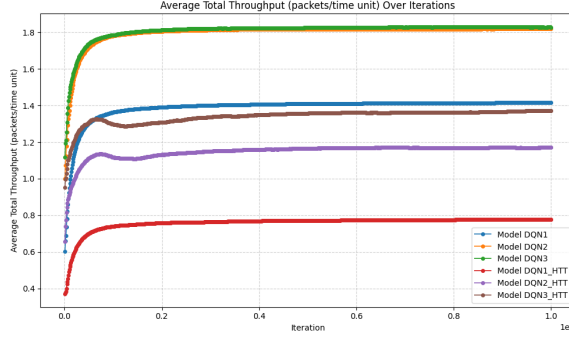
This section compares the proposed solution with the HTT (Harvest-Then-Transmit) scheme, which allows only energy harvesting and active transmission without using backscatter communication. The comparison highlights the impact of incorporating backscatter on the overall system performance.

Figure 4.15(a) and Figure 4.15(b) shows that overall without having access to backscatter, HTT scheme perform significantly worse than the proposed method. The reason could be because when jammer is at high level of power using rate adaption cant transmit as much packets or none at all while backscatter could make use of that to transmit with higher number of packets.

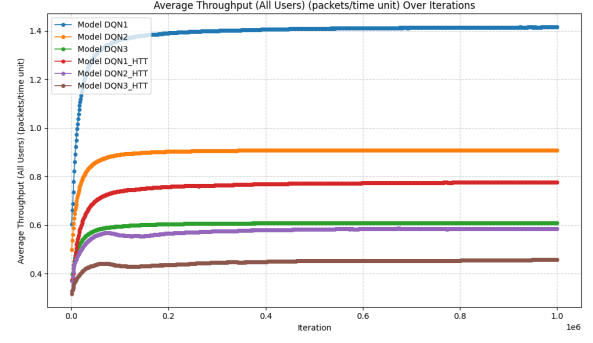
As a result, Figure 4.15(c) shows that the HTT scheme experiences a higher packet loss ratio compared to the proposed method, due to its limited ability to transmit packets. Additionally, it causes a higher packet loss by transmission, since the proposed method performs more frequent transmissions than HTT.

This section is simulated using a 2-agent system over 2×10^5 iterations to ensure convergence rate.

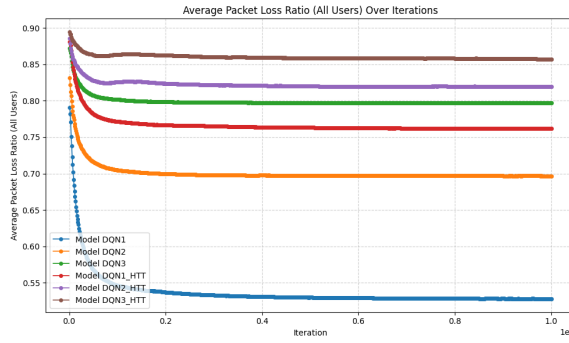
Figure 4.16 shows that, in general, the proposed method has better performance.



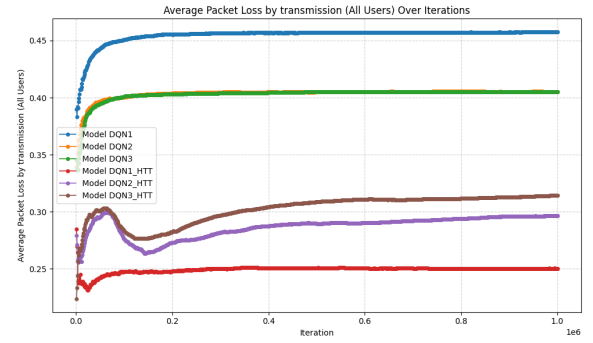
(a) Comparison of Convergence Rates of the Proposed Method vs. HTT Scheme.



(b) Comparison of Average Throughput Convergence of the Proposed Method vs. HTT Scheme.



(c) Comparison of Packet Loss Ratio of the Proposed Method vs. HTT Scheme.



(d) Comparison of Packet Loss Ratio by Transmission of the Proposed Method vs. HTT Scheme.

Figure 4.15. Performance Comparison of Proposed Method vs. HTT Scheme Over 1M Iterations.

However, as the probability of the jammer being idle increases, the throughput of both methods becomes more similar. This is because the proposed method can no longer rely heavily on backscatter and instead switches to more active transmissions, which aligns its behavior more closely with the HTT scheme, resulting in similar performance.

Figure 4.17 shows a similar trend to the previous figure, where the proposed method initially achieves higher throughput and thus experiences a lower packet loss ratio due to reduced data overflow. However, as the jammer idle probability increases, the performance of both methods begins to align. Figure 4.18 shows that both methods exhibit increasing and decreasing trends in packet loss by transmission. The HTT scheme has a lower packet loss by transmission because it transmits less frequently, and most of the overall packet loss is caused by data overflow rather than failed transmissions.

* Proposed Solution vs. Fixed Agent TDMA

The results below show the learning capability of the current method compared to

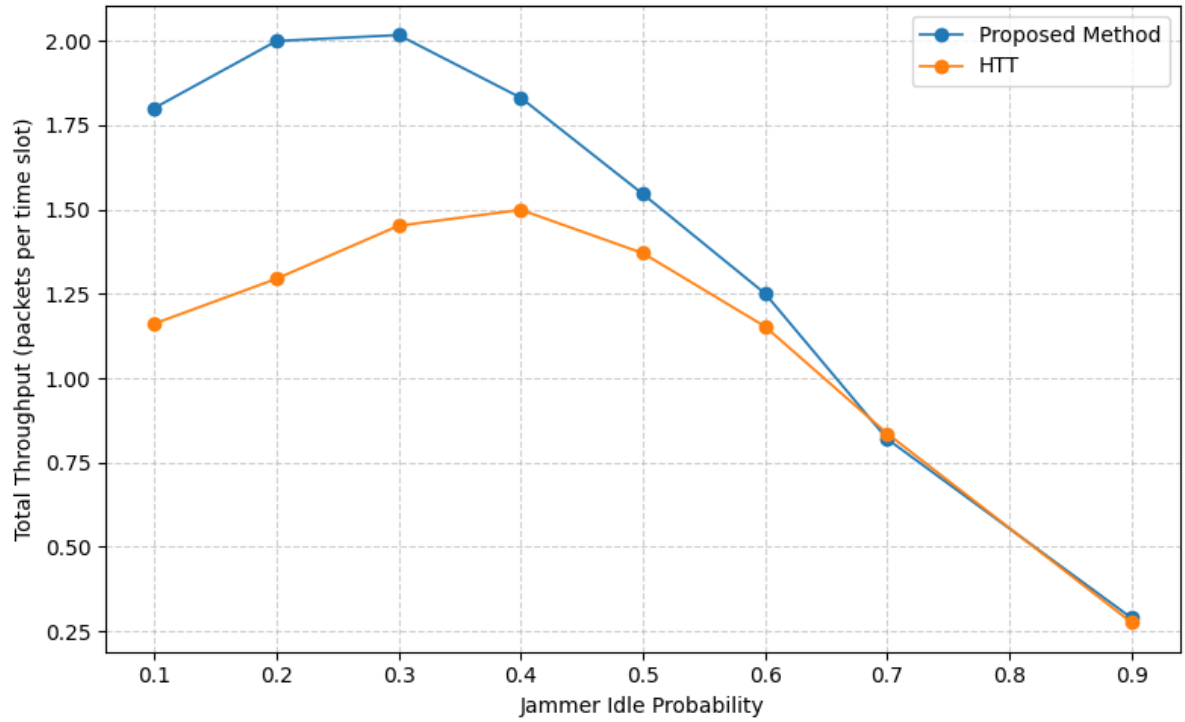


Figure 4.16. Convergence rates of the proposed method and HTT scheme under varying jammer idle probabilities.

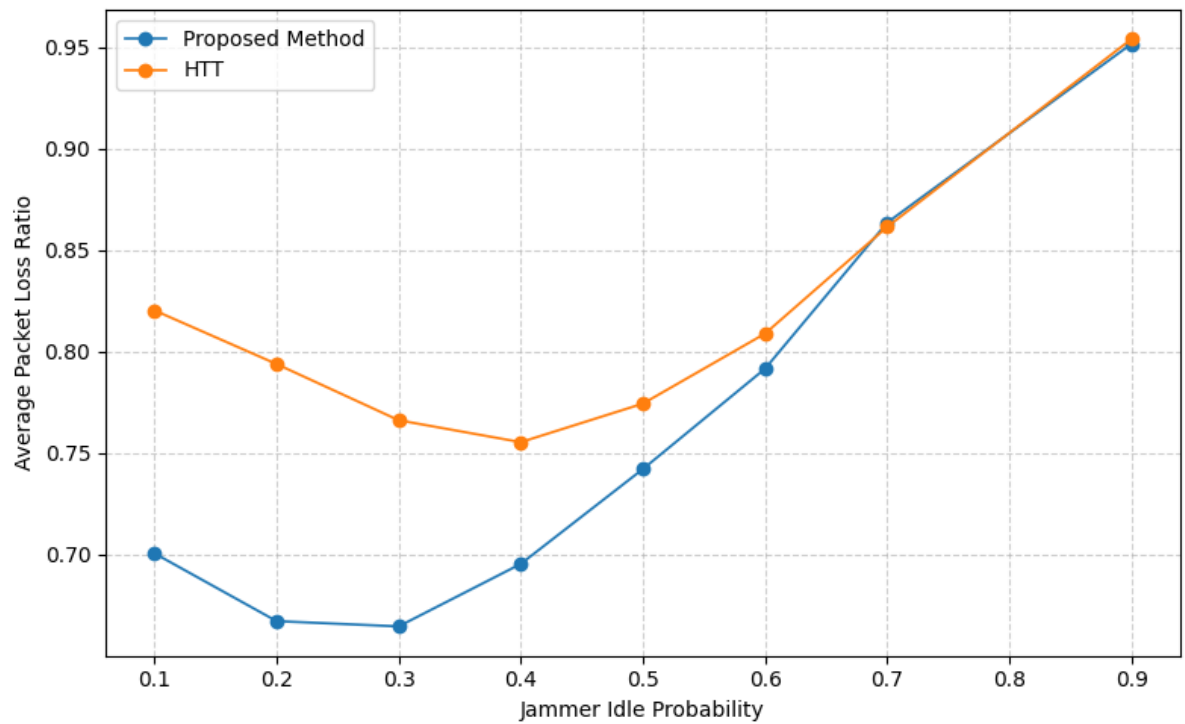


Figure 4.17. Packets loss ratio of the proposed method and HTT scheme under varying jammer idle probabilities.

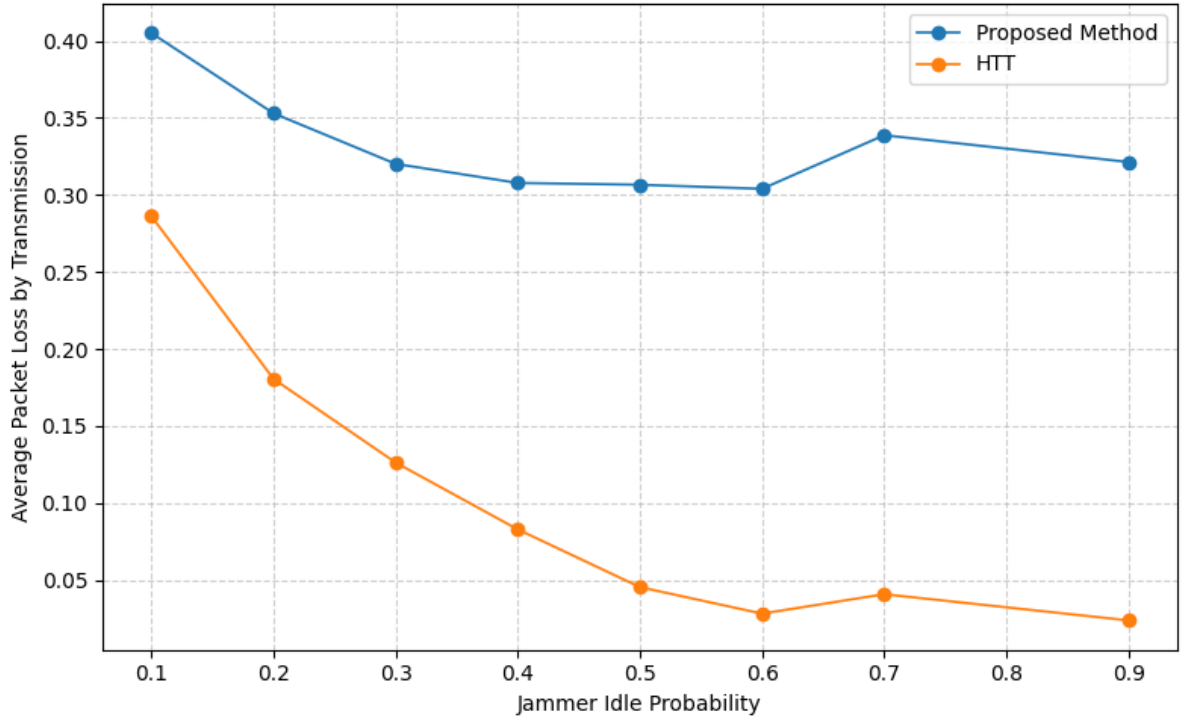


Figure 4.18. Packets loss ratio by transmission under varying jammer idle probabilities for the proposed method and HTT scheme.

using a fixed agent TDMA, where the primary agent can transmit and the others remain idle or harvest, with no transition to another agent if the current agent doesn't transmit. As shown in the three figures—Figure 4.19, Figure 4.20(a), and Figure 4.20(b)—the current method exhibits better learning performance than the fixed agent approach. Fixed agents have a slower convergence rate and a worse loss ratio compared to the current method. This is because, with a fixed agent, when the primary agent does not transmit, the transmission capability of other agents is not utilized, resulting in the loss of some transmission packets.

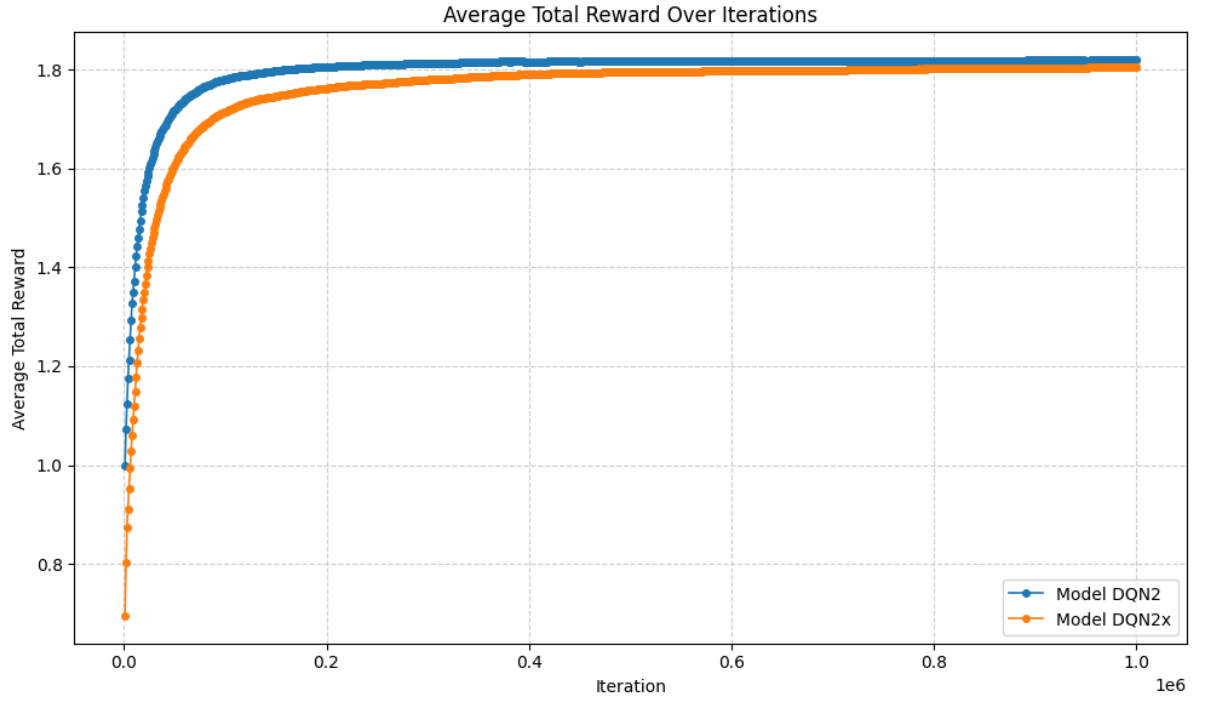
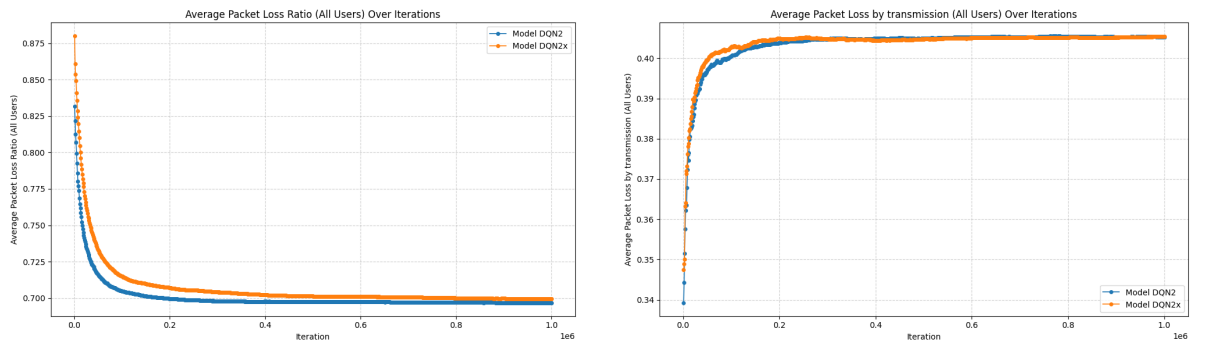


Figure 4.19. Comparison of Convergence Rates of the Proposed Solution vs. Fixed Agent TDMA.



(a) Comparison of Loss Ratio of the Proposed Solution vs. Fixed Agent TDMA. (b) Comparison of Loss Ratio By Transmission of the Proposed Solution vs. Fixed Agent TDMA.

Figure 4.20. Loss Ratio Comparison of the Proposed Solution vs. Fixed Agent TDMA.

CONCLUSIONS

This thesis proposed and evaluated a novel cooperative multi-agent deep reinforcement learning (MADRL) anti-jamming method for backscatter communication networks, addressing the critical challenge of jamming attacks in energy-constrained wireless systems. By modeling each transmitter as an intelligent agent operating under a Time Division Multiple Access (TDMA) scheme, the proposed method leverages Deep Q-Network (DQN) techniques to enable agents to learn cooperative strategies that optimize throughput and resilience against dynamic jamming conditions. The key findings, contributions, limitations, and future research directions are summarized below.

Key Findings

The simulation results demonstrate that the proposed cooperative multi-agent DQN approach significantly outperforms traditional Q-learning and greedy anti-jamming strategies in terms of convergence rate, average rewards, and packet loss ratio. Specifically, the DQN-based method achieved average rewards of approximately 1.8 packets per time slot, while Q-learning requires an increased number of agents to reach a similar level, with a higher risk of packet loss—indicating the superior learning efficiency of DQN in multi-agent scenarios. The packet loss ratio, both from queue overflow and transmission errors, was notably lower with DQN, underscoring its effectiveness in adapting to jamming through energy harvesting, backscattering, and rate adaptation. Furthermore, the analysis of varying agent group sizes revealed that larger groups accelerate early learning due to increased energy harvesting opportunities, although they may face higher packet loss due to queue overflow in later stages.

In the comparison between multi-agent and single-agent systems, the proposed method demonstrates faster convergence and lower packet loss ratios. This is attributed to dynamic agent scheduling in TDMA, which enables better utilization of transmission opportunities across all agents. Such flexibility mitigates the inefficiencies of fixed-agent systems, where non-transmitting agents remain idle—resulting in higher packet loss due to queue overflow. The multi-agent DQN system further outperforms the single-agent counterpart, achieving up to 1.8 packets per time slot, compared to 1.4 for single-agent

DQN and 1.35 for Q-learning. This improvement is enabled by coordinated energy harvesting and transmission strategies. However, this advantage comes with a trade-off: the multi-agent system exhibits a higher overall packet loss ratio due to increased queue overflow from longer waiting times inherent in TDMA scheduling.

Beyond these initial insights, the proposed DQN method exhibits a better improvement in throughput compared to the HTT scheme, as evidenced by the total throughput metrics across 1M iterations. This superiority stems from the integration of backscatter communication, which allows the proposed method to transmit packets under high jamming power levels where HTT fails, leveraging harvested energy effectively. The comparison with the HTT scheme also highlights a reduction in packet loss ratio, attributed to the DQN's ability to dynamically switch between backscatter and rate adaptation strategies based on jamming conditions. However, as the jammer idle probability (ν) increases from 0.1 to 0.9, the performance gap narrows, with both methods converging toward similar throughput levels due to reduced reliance on backscatter and increased active transmission opportunities.

These findings validate the potential of multi-agent deep reinforcement learning (MADRL) in enhancing the robustness of backscatter communication networks against jamming attacks. The proposed method's ability to adapt to dynamic jamming environments, leverage backscatter, and coordinate multi-agent actions underscores its scalability and efficiency, paving the way for its application in next-generation low-power wireless systems.

Limitations

Despite its strengths, the proposed method has certain limitations. Increasing the number of agents leads to higher packet loss, so the agent count must be chosen carefully. The reliance on a single-channel TDMA system restricts the scalability of the approach in multi-channel environments, where concurrent transmissions could further enhance throughput. Additionally, uniform sampling in the experience replay buffer may delay the incorporation of recent transitions, potentially slowing adaptation to rapidly changing jamming patterns. The computational complexity of training multiple DQN agents also poses challenges for real-time implementation on resource-constrained devices.

Future Research Directions

Future work can address these limitations by exploring several promising directions. First, incorporating prioritized experience replay or advanced sampling techniques could enhance the DQN's ability to adapt to dynamic jamming strategies. Second, investigating hybrid reinforcement learning approaches could further improve performance and cooperation among agents. For instance, combining DQN with actor-critic methods or transfer learning could reduce training time and improve performance against adaptive jammers. Finally, exploring advanced multi-agent reinforcement learning algorithms like QMIX could enhance cooperative learning by leveraging a centralized mixing network to better coordinate actions among agents, ensuring more efficient anti-jamming strategies in dynamic environments. These advancements would further strengthen the resilience of next-generation wireless communication systems.

In conclusion, this thesis demonstrates the efficacy of cooperative multi-agent deep reinforcement learning in combating jamming attacks in backscatter communication networks. By providing a robust, adaptive, and scalable anti-jamming solution, this work paves the way for more secure and efficient low-power wireless systems, contributing to the broader adoption of intelligent technologies in wireless communications.

REFERENCES

- [1] D. T. Hoang, N. Van Huynh, D. N. Nguyen, E. Hossain, and D. Niyato, *Deep Reinforcement Learning for Wireless Communications and Networking: Theory, Applications and Implementation*, Wiley, 2023, pp. 37–163.
- [2] N. Van Huynh, D. N. Nguyen, D. T. Hoang, and E. Dutkiewicz, “Jam Me If You Can: Defeating Jammer With Deep Dueling Neural Network Architecture and Ambient Backscattering Augmented Communications,” *IEEE Journal on Selected Areas in Communications*, vol. 37, 2019, pp. 2603–2620.
- [3] F. Yao and L. Jia, “A Collaborative Multi-Agent Reinforcement Learning Anti-Jamming Algorithm in Wireless Networks,” *IEEE Wireless Communications Letters*, vol. 8, no. 4, 2019, pp. 1024–1027.
- [4] K. Grover, A. Lim, and Q. Yang, “Jamming and anti-jamming techniques in wireless networks: A survey,” *International Journal of Ad Hoc and Ubiquitous Computing*, vol. 17, no. 4, 2014, pp. 197–215.
- [5] H. Ding, Y. Niu, Q. Zhou, and X. Peng, “A novel intelligent anti-jamming communication algorithm based on proximal policy optimization,” *Physical Communication*, vol. 65, no. 18, May 2024, pp. 102366.
- [6] S. B. Janiar and P. Wang, “Intelligent anti-jamming based on deep reinforcement learning and transfer learning,” *IEEE Transactions on Vehicular Technology*, vol. 73, no. 6, June 2024, pp. 8825–8834.
- [7] V. Mnih, K. Kavukcuoglu, D. Silver, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, 2015, pp. 529–533.